

APUNTESWEB · INGENIERÍA INFORMÁTICA



Tercer curso · 1.º cuatrimestre · 6 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Instalación de Linux Debian

IMPORTANTE

Estos apuntes están bien para entender ciertos conceptos complejos. Sin embargo si quieres sacar nota en el examen (ya te aviso que complicado), yo complementaré con los del profe porque tiene más ejemplos e información más concisa.

1.1 Tipos de Instalación

A la hora de instalar un sistema, tenemos que tener en cuenta el tipo de funciones que va a desempeñar.

- **Sistema de escritorio:** usado en tareas rutinarias
- **Estación de trabajo:** sistema de alto rendimiento, generalmente orientado a una tarea específica.
- **Servidores:** ofrecen servicios a otras máquinas de la red
 - **Servicios de disco:** acceso a ficheros a través de FTP, servicio de disco transparente a través de NFS o Samba
 - **Servicios de aplicaciones,** por ejemplo, terminales, conexión remota (telnet, ssh) ...
 - **Servicios de directorio,** por ejemplo, LDAP, que almacena credenciales de usuarios, directorios, etc.
 - **Servicios de bd**
 - **Servicios de impresión**
 - **Servicios de red**

1.2 Instalación

Primero descargamos la imagen de CD de instalación por red de la versión seleccionada (`debian-versión-amd64-netinst.iso`).

NOTA

Esta explicación antes de la imagen no entra en el final, pero joder yo creo que son cosas que no todo el mundo tiene claro y veo importantes. Seguramente meta más cosas que vea interesantes pero ya pondré algo como esto para saber si entra o no. Me jode q una asignatura como esta no explique la mitad de conceptos como se debería.

Un archivo **.iso** (o imagen ISO) es una **copia digital exacta** de un disco óptico (CD, DVD o Blu-ray). Imagínatelo como una "caja virtual" que contiene todos los archivos de instalación del sistema operativo y, lo más importante, la estructura necesaria para que tu ordenador pueda "arrancar" (bootear) desde ese archivo.

Cuando ves una opción llamada **Netinst** (Network Installer) o "Imagen de instalación por red", se refiere a una **ISO minimalista**. A diferencia de una imagen estándar, esta ISO contiene **solamente lo estrictamente necesario** para:

1. Arrancar el ordenador.
2. Detectar tu hardware básico (tarjeta de red, disco duro).
3. Conectarse a internet.

Una vez que arranca el instalador, este **descarga** el resto del sistema operativo (el entorno gráfico, las aplicaciones, el navegador, etc.) directamente desde los servidores oficiales de Linux *durante* el proceso de instalación.

Tiene ciertas ventajas, como que siempre va a estar actualizado, ocupa poco espacio y es personalizable, si no te quieres bajar algunos paquetes puedes desmarcarlos y quitarte bastante morralla.

Sin embargo como no tengas internet estás jodido y la instalación obviamente va a ser más lenta si usas internet.

Además ya para llegar a este paso deberías saber cómo bootear un SO. Eso lo tengo explicado en un repositorio llamado HyprDebian en mi perfil de github.

Cuando lanzas el iso te salta esta pestaña y empieza la magia.

1.2.1 Sigüientes pasos en la instalación

Seleccionas el idioma, localización y teclado.

Configuración de la red, por defecto, intenta configurarla por DHCP. Si no lo consigue, pasa a configuración manual (indicar máscara, pasarela y DNSs)

Para que nos entendamos, configurar la red es decirle a tu ordenador quién es, dónde está y a quién debe preguntar para salir a internet.

Por si no te acuerdas de redes, **DHCP** es un sistema de **configuración automática**. Cuando tu ordenador arranca, grita a la red "Soy nuevo, alguien me da una dirección", y el servidor DHCP (normalmente el router de casa) responde y le entrega todos los datos necesarios (IP, máscara, puerta de enlace, DNS) automáticamente.

Si esto falla estamos jodidos porque la tienes que configurar tu a mano completando esos campos:

- **IP**, matrícula única que identifica a tu máquina dentro de la red

- **Máscara de subred**, le indica a tu ordenador qué direcciones IP son sus vecinos (red local) y cuales están fuera (internet). Por lo que define el tamaño de tu red local.
- **Pasarela o Puerta de enlace**, es la dirección IP del **router**. Cuando tu pc quiere entrar a una web que no está en tu casa, no sabe como llegar, la pasarela es a quien le entrega el paquetes de datos para que lo envíe fuera a Internet, si pones mal esto, vas a tener red local pero no acceso a internet.
- **DNS**, Domain Name System. Son servidores que traducen nombres humanos a direcciones IP. Cuando escribes `google.com` el servidor DNS le dice a tu ordenador: eso es la IP `142.250.200.46`

Después le ponemos un nombre a la máquina e indicar el dominio. El nombre de la máquina (**hostname**) es como te llamas tú o como quieres que se le llame a tu PC para diferenciarlo de otros en la misma red (bastante más cómodo que andarle metiendo la ip para comunicarse con el). El dominio es como tu apellidos, si vas a usar tu pc para ti no tiene mucho sentido ponérselo. Sin embargo si estas en una empresa es bastante útil.

Por último se fija el password del superusuario (root) y se crea un usuario no privilegiado.

1.2.2 Cuenta del superusuario

Es un usuario especial que actúa como administrador del sistema. Tiene acceso a todos los ficheros y directorios del sistema, puede crear otros usuarios o eliminarlos. Además puede tanto borrar como instalar software del sistema o aplicaciones. Puede detener cualquier proceso que se está ejecutando en el sistema. Hasta puede detener y reiniciar el sistema.

Para cambiarnos de un usuario normal a **superusuario** usamos `su -`, dónde `su` es un comando que nos permite cambiar de usuario `su [opciones] [-] username`, si no especificamos `username` pasamos a **root** (**superusuario**).

1.2.3 Elección de contraseña

Lo suyo es escoger una contraseña segura, si eres tonto y metiste como contraseña `asdf` y quieres proteger a tu ordenador un mínimo siempre la puedes cambiar gracias al comando `passwd`:

1.2.4 Continuación de la instalación

En una instalación por red los paquetes se traen de un repositorio remoto a través de http o ftp (ftp no se usa normalmente pero aún hay algunos servidores de debian que lo usan).

Seleccionamos el huso horario. Y realizamos el particionado del disco (modo guiado o manual).

1.2.5 Particionado del disco

Podemos optar por instalar todo el sistema en una sola partición, aunque no es nada recomendable (a ver en verdad es recomendable si no quieres romperte la cabeza o arriesgarte a joder todo el disco). Lo más recomendable es instalar diferentes directorios del sistema en diferentes particiones.

Para entender bien lo que es una **partición** imagínate que tu disco duro es una **nave industrial vacía**. Una partición es levantar paredes para dividir esa nave en **habitaciones separadas**. Físicamente sigue siendo el mismo edificio pero lógicamente, lo que ocurre en una habitación no afecta al resto. Si se inunda el baño (una partición), no se te van a mojar los muebles del salón.

Los **directorios** son simplemente las carpetas donde el sistema operativo organiza los archivos. Los más importantes en linux son:

- `/`: es la base de todo, aquí van los archivos vitales
- `/home`: aquí van los documentos personales
- `/var`: aquí se guardan cosas que cambian mucho, como los registros de errores o web si tienes un servidor
- y así otros muchos directorios que se explican más adelante

Se recomienda poner directorios en diferentes particiones porque:

- Caso de `/home`:
 - Si tienes todo en **una partición** y tu linux se rompe o quieres instalar una versión más nueva, al formatear el disco para instalar el sistema, **borrar también tus fotos y documentos personales** porque todo está en el mismo saco.
 - Si lo tienes **separado**: tienes una partición para el sistema y otra diferente para tus datos, por lo que puedes borrar, formatear y reinstalar el sistema operativo 50 veces en la primera partición y tus fotos en `/home` siguen intactas.
- Caso de `/var`: Imagínate que un programa se vuelve loco y empieza a escribir errores en un archivo de texto sin parar
 - **Todo en una partición**: el archivo crece hasta llenar el disco y el ordenador se bloquea
 - `/var` **separado**: el archivo llena `/var` pero `/` sigue con espacio y el ordenador sigue funcionando.

1.2.6 Fylesystem Hierarchy Standard

La entrada principal

`/` todo pende de este directorio llamado **root o raíz**, se puede pensar como que vestíbulo principal y el resto de carpetas son pasillos que salen de este vestíbulo.

Las Herramientas (programas)

En lugar de tener una carpeta por programa, Linux separa los archivos en diferentes "cajas" según quién los usa y para qué:

- `/bin/` (Binaries): es la **caja de herramientas de emergencia**. Contiene lo mínimo para que el sistema funcione (como `ls` para listar archivos o `cp` para copiar). Si se rompe todo, esto debe estar disponible.
- `/sbin/` (System Binaries): son herramientas exclusivas para el **administrador del sistema**. Aquí están los comandos para formatear discos, configurar redes, etc. Un usuario normal no los puede usar.
- `/usr/` (User System Resources): aquí es donde vive el 90% del software "normal". Es como el almacén general.
 - `/usr/bin`: aquí están chrome, libreoffice, etc
 - `/usr/local`: aquí va lo que instalar tú manualmente para no mezclarlo con lo que trae el sistema
- `/opt` (Optional): es para software extraño o comercial que no sigue las reglas de Linux. Son aplicaciones que requieren un subdirectorio separado del resto.

La sala de máquinas y configuración

- `/boot`: es el **botón de encendido**. Contiene los mapas y las instrucciones exactas para arrancar el ordenador
- `/etc`: es el **panel de control**. Aquí no hay programas, solo archivos de texto con configuraciones. Si por ejemplo quieres cambiar la IP, está en un archivo en `etc`. Scripts de configuración.
- `/lib`: son las **piezas de repuesto compartidas**. Los programas de `/bin` necesitan estas piezas para funcionar. Librerías.

Los despachos

- `/home`: es tu **despacho personal**. Aquí hay una carpeta con tu nombre. Es el único sitio donde guardar fotos, descargar y documentos libremente, fuera de aquí no tienes permiso para escribir. Es el directorio donde están los directorios de los distintos usuarios.
- `/root`: **despacho blindado del superusuario**.

El almacén dinámico y temporal

- `/var` (variable): piensa en esto como el **buzón de correos y el libro de reclamaciones**. Aquí se guardan cosas que crecen y cambian constantemente
- `/tmp` (temporal): es la **pizarra borrrable**. Los programas escriben notas aquí mientras trabajan cuando reinicias el ordenador, todo lo que hay aquí se borra.
- `/srv`: datos de servicios proporcionados por el sistema

Las conexiones externas

- `/media`: es automático. Si metes un usb, el sistema crea una carpeta aquí automáticamente para que veas el contenido. Es el punto de montaje para medios removibles
- `/mnt`: es manual. Se usa históricamente para que el administrador "pegue" discos duros o redes temporalmente. Es el punto de montaje para otros sistemas temporales (por ejemplo, el montaje de una unidad de red, de una partición de otro sistema operativo, etc)

Los fantasmas (directorios virtuales)

Son complejos de entender puesto que **no existen en el disco duro**. Son ilusiones creadas por el sistema operativo en la memoria RAM.

- `/proc` y `/sys`: son una ventana al cerebro del ordenador en tiempo real, si entras en `/proc/cpuinfo`, no lees un archivo guardado, lees directamente lo que el procesador le está diciendo al SO en ese milisegundo. `/proc` contiene información del sistema y `/sys` contiene información de dispositivos
- `/dev` (devices): en linux, **todo es un archivo**. Tu ratón es un archivo, tu disco duro es un archivo. Es un directorio que contiene pseudoficheros de acceso a periférico

1.2.7 Esquemas de particionamiento

Dependiendo del tipo de sistema que queramos montar podemos escoger diferentes esquemas de particionamiento:

NOTA

He metido información de más que considero interesante, si solo quieres aprobar con que sepas que particiones se recomiendan por esquema puedes continuar.

Máquina de escritorio (Tu pc personal)

El objetivo es la simplicidad y proteger tus datos, por lo que se recomiendan 3 particiones:

- `/` raíz, aquí va el Sistema Operativo
- **swap** (área de intercambio): cuando la RAM se llena, el sistema coge aquellos archivos que menos estás usando en ese momento para liberar espacio y los mete al swap, y cuando los vuelve a necesitar los saca del swap. Es un proceso mucho más **lento**, es un simple **salvavidas** que evita que un programa que usa mucha memoria se cierre de golpe o el PC se congele. **Si quieres hibernar**, todo lo que hay en la RAM se copia al **swap** y cuando quieres volver a arrancarlo se vuelve de nuevo a la RAM. La mítica frase de **tienes que tener el doble de swap que de RAM** pues joder tiene matices porque claro si tienes una castaña de RAM de 4 gb la vas a llenar muchas veces entonces pues el lógico tener el doble de swap, pero si tienes una RAM de

16 GB intenta llenarla fenómeno, por eso meter 32 gb de swap con poco disco es de no ser muy listo.

- `/home`, esto es lo crucial. Al tener tus datos personales en una **habitación aparte**, si mañana te da por romper tu sistema operativo no vas a perder todos tus datos, puedes formatear la partición `/` y reinstalar el sistema sin tocar la partición.

Sistema multiusuario

El objetivo es la seguridad y que el servidor nunca se apague. Hay que evitar que uno de los múltiples usuarios sea capaz de cargarse todo el sistema, por ello hay múltiples particiones:

- Separar `/var` y `/tmp`, imagina que un usuario ejecuta un programa mal hecho que empieza a generar errores infinitos o a descargar archivos basura sin parar. **Si todo está junto** se llena el disco entero y el sistema operativo se queda sin espacio y el **servidor colapsa**. Si tenemos `/var` separado el archivo de errores solo llena la partición, sin afectar a `/` el servidor seguirá funcionando.
- Separar `/usr` sirve para protegerse de virus y hackers, puesto que es donde viven los programas ejecutables. Si lo pones en una partición aparte, puedes activarla como **solo lectura**, esto implica que una vez instalado el sistema, pones un candado a esa partición. Aunque entre un virus o un hacker, no podrá modificar los archivos ejecutables para infectarlos, porque estará prohibido. Tus usuarios no podrán bajarse apps de forma libre.

1.2.8 Particionamiento durante la instalación

Dos opciones:

- Particionamiento guiado (con o sin LVM, LVM ya lo explico más adelante), donde se selecciona el tamaño de las particiones de manera automática
- Particionamiento manual, con control del número y tamaño de las particiones

1.2.9 Particionamiento manual

Seleccionamos el disco a particionar y crear nueva tabla de particiones

Creamos una nueva partición indicándole el tamaño, el tipo (primaria o lógica) y la localización (comienzo o final). Con la **BIOS** clásica puede haber 4 primarias o 3 primarias y una extendida, que a su vez se puede dividir en varias lógicas. Con **UEFI** se pueden crear hasta 128 primarias por disco.

Si no sabes que es **BIOS** o **UEFI** es simplemente un **firmware** (un mini-software) que vive en un chip de la placa base de tu ordenador. Es lo primero que se despierta al pulsar el botón de encendido, comprueba que la RAM y el procesador funcionan y luego busca un disco duro para arrancar el SO.

- **BIOS** es el sistema **antiguo**
- **UEFI** el **moderno**

1.2.10 Sistemas de ficheros

Linux soporta varios sistemas de archivos:

- **ext4:** Fourth EXTended filesystem, es el sistema de ficheros estándar en Linux
 - Es un sistema de ficheros transaccional (como una bd)
 - Tiene características que le permiten reducir la fragmentación
 - Puede trabajar con discos y ficheros de gran tamaño.
 - Las opciones pueden configurarse con el comando `tune2fs`
 - Las anteriores versiones `ext3` y `ext2` siguen estando disponibles

NOTA

btrfs no entra pero yo de ti si fuese a montar mi SO usaría este porque es la versión moderna

- **btrfs:** posible sucesor de ext4, pues presenta características avanzadas que además de mejorar el rendimiento, van dirigidas a la gestión y seguridad del almacenamiento:
 - Gestiona de manera integrada el almacenamiento, pues incluye funciones que antes formaban parte del sistema de ficheros, del controlador RAID y del gestor lógico de volúmenes LVM
 - Hace uso extensivo de *copy-on-write*, si varios recursos son idénticos se devuelve un puntero a un único recurso; en el momento en que se modifica una “copia” del recurso, se crea una copia auténtica para prevenir que los cambios sean visibles a las demás copias.
 - Permite *snapshots* de solo lectura o modificables
 - Etc
- **NFTS, exFAT:** usados por Windows en ordenadores domésticos soportados también por linux
- **ReFS:** usado por windows para empresas, no disponible de forma nativa para windows

1.2.11 Últimos pasos de la instalación

Debemos seleccionar el **mirror** desde el que descargar el software. Existen varios repositorios de paquetes Debian, y debemos elegir el más cercano. Introducir la información del proxy.

Después seleccionas los paquetes de software a instalar y por último instalas el gestor de arranque.

Si quieres entender lo que es un **mirror** piensa que debian es una **fábrica gigante de USA** donde se fabrica el software. Si millones de personas de todo el mundo se intentase bajar el SO de esa fabrica el mismo tiempo, sería todo lentísimo y la fábrica colapsaría. Un **mirror** es un servidor que contiene una copia exacta (un espejo) de todo lo que hay en el servidor principal. Hay miles por todo el mundo, por ello escoges el más cercano a tu país.

Un **proxy** es un intermediario.

- Sin Proxy: Tu PC -> Internet
- Con Proxy: Tu pc -> Servidor proxy (revisa quién eres y qué pides) -> Internet

1.2.12 Instalación del gestor de arranque

Podemos tener diferentes distribuciones de Linux y Windows en el mismo ordenador, cada una con sus correspondientes particiones. El gestor de arranque (cargador o *bootloader*) nos permite seleccionar el SO a arrancar.

- Las distribuciones **Linux** usan el cargador **GRUB** (GRand Unified Bootloader)
- Cuando el sistema se inicia, la **BIOS** carga el gestor de arranque que nos permite seleccionar el SO y a continuación transfiere el control al programa de inicio del correspondiente SO (localizado en /boot)

Tenemos dos posibilidades a la hora de instalarlo:

- Instalarlo en el **MBR** o **GTP** del primer disco:
 - El MBR contiene información sobre las particiones del disco y un pequeño código del gestor de arranque. MBR se almacena solo en el primer sector del disco, como se joda esta parte del disco te jodiste.
 - El GTP es el nuevo estándar que sustituye al anterior, más fiable y asociado a los sistemas UEFI. GTP Crea múltiples copias redundantes a lo largo de todo el disco y su nombre hace referencia a que a cada partición se le asocia un identificador global único
- En caso de que tengamos otro cargador en el MBR o GTP, el gestor de arranque GRUB puede instalarse en el primer sector de la partición Linux que contenga /boot

Debemos instalar al menos un gestor de arranque, en caso contrario no podríamos arrancar el sistema operativo. Si tenemos varios sistemas operativos en la misma máquina basta con instalar el gestor de arranque para uno de ellos (usualmente el del último sistema que instalemos).

Podemos configurar GRUB para evitar que sea modificado el menu de arranque. En concreto, podemos usar una contraseña para limitar:

- la modificación de los parámetros iniciales
- el acceso a determinadas imágenes
- el acceso a opciones avanzadas

Una vez acabada la instalación **reiniciamos**.

1.2.13 Logical Volume Management (LVM)

LVM es una de las tecnologías más potentes de Linux, pero también abstracta.

Imaginate que tienes dos discos duros, son como dos barras de plastilina dura y seca. Sin LVM, si necesitas hacer una figura (partición) y se te acaba la plastilina de la barra 1, **no puedes** coger un trozo

de la barra 2 y pegárselo. Son independientes. Si tu carpeta de usuario se llena, tienes que comprar un disco más grande y moverlo todo, es rígido.

LVM coge esos discos duros, los tritura y crea una **masa gigante y uniforme de plastilina**. Ahora, el sistema operativo no le importa de qué disco viene el material, solo ve una gran reserva de espacio para usar.

Los 3 niveles de LVM:

- **PV - Physical Volume.** Son los discos duros, particiones,... Las barras de plastilina originales (llamadas por norma general `sda`, `sdb`). Para usar LVM, primero tienes que marcar tus discos como listos para ser usados en LVM.
- **VG - Volume Group.** En una agrupación de PV. Es la bola gigante de plastilina. Puedes coger tus PVs y los mezclas en un solo grupo (`diskvg`). Ahora tienes un montón de espacio libre sumado. Ahora el sistema en vez de ver dos discos pequeños ve uno grande.
- **LV - Logical Volume.** Particiones lógicas sobre las que se montan los sistemas. Son las **figuras que moldeas** sacando material de la bola grande. Creas particiones virtuales. Creas un LV para el sistema, otro para datos, etc. Si tu figura (LV) se queda pequeña, simplemente coges más plastilina de la bola grande (VG) y la haces más grande al instante, sin reiniciar. Y si te quedas sin espacio simplemente compras un nuevo disco y lo añades al VG.

1.2.14 Como crear un sistema LVM

Crear los PV (Preparar los ingredientes): reservar espacio para `/boot` fuera de LVM. El gestor de arranque (GRUB) es muy básico. A veces le cuesta "entender" la complejidad de LVM. Por seguridad, se crea una partición pequeña normal y corriente para el arranque (`/boot`), y el resto del disco se marca como PV (plastilina para el sistema).

Crear un grupo de volumen (Hacer la masa): Incluye `sda2` y `sdb1`. Aquí estás fusionando el espacio del primer disco y del segundo disco en un único almacén gigante llamado `diskvg`.

Crear los volúmenes lógicos (Hacer las figuras): Ahora que tienes el almacén `diskvg`, empiezas a cortar trozos virtuales:

- Un trozo para `/` (`rootlv`).
- Un trozo para `/usr` (`usrlv`).
- Un trozo para `/var` (`varlv`).

Cifrar (Poner caja fuerte): Como LVM es una capa de software, puedes decirle al sistema: Oye, este volumen lógico (`varlv`) quiero que vaya cifrado. El sistema te pedirá contraseña cada vez que intente acceder a esa "figura" de plastilina.

Asignar sistema de ficheros (Pintar): Finalmente, a esos volúmenes lógicos les das formato (`ext4`, `xf`s, etc.) para poder guardar archivos, igual que harías con una partición normal.

1.3 Arranque del sistema

Al arrancar el ordenador se realizan las siguientes operaciones:

BIOS/UEFI (El vigilante de seguridad)

El vigilante (firmware de la placa base) llega al edificio oscuro y empieza su turno:

- **Chequeo (POST):** Comprueba que el edificio no se cae (mira si la RAM, el teclado y la pantalla funcionan).
- **Decisión:** Mira su lista de tareas para ver el orden de arranque. ¿A quién le doy las llaves? ¿Al disco duro, al USB o a la red?
- **Lectura del Disco:** Encuentra el disco duro principal. Aquí cambia según el sistema:
 - **Si es MBR (Antiguo):** El vigilante va a la puerta de entrada (primer sector) y despierta a quien esté allí durmiendo (ejecuta el código ciego del MBR).
 - **Si es GPT (Moderno):** El vigilante **lee un mapa en la pared** (La Tabla de Particiones GUID). El mapa le dice: "El Recepcionista está en la oficina 1". El vigilante va allí y lo busca.

Gestor de Arranque - GRUB (El recepcionista)

Una vez que el vigilante ha localizado y despertado al recepcionista (GRUB), el vigilante se retira.

- **El Menú:** El trabajo del recepcionista es saludarte y mostrarte un menú: "¿Qué sistema operativo quiere usar hoy?".
- **La Elección:** Si no tocas nada, elige la opción por defecto. Si eliges tú (ej. Linux), el recepcionista mira en su lista dónde están las llaves de ese sistema.
- **El Traspaso:** Va al almacén (/boot), saca al **Gerente (Kernel)** y lo despierta (carga el kernel en la memoria RAM), cediéndole el control total del edificio.

El kernel e initframs (El gerente y su mochila)

El **kernel** es el cerebro del sistema, el gerente. Pero tiene un problema. Imagina que las herramientas para abrir las puertas del centro comercial están guardadas dentro del centro comercial:

- El Kernel necesita acceder al disco duro real para cargar los controladores (drivers) del disco duro.
- Pero no puede leer el disco duro porque aún no ha cargado los controladores.

La solución: `initframs` (la mochila de emergencia). Para solucionar esto, el Kernel no va al disco duro real todavía. Carga en la memoria RAM un **disco falso y temporal** (virtual) llamado `initframs`.

`initframs` contiene una partición raíz virtual y un programa `init` que permite arrancar todo.

- Es como si el gerente trajera una **mochila de casa**.

- Dentro de esa mochila (RAM) tiene las herramientas mínimas para entender como funciona el disco duro, desbloquear el disco si está cifrado y entender LVM

Una vez que usa las herramientas de la mochila para abrir el disco duro real, tira la mochila y entra al sistema real.

`systemd` (El jefe de la planta)

Una vez que el Kernel ha entrado en el disco real, llama al primer proceso real: `systemd` (o `init`).

A partir de aquí, el kernel, se dedica a gestionar el hardware, y `systemd` se encarga de organizar a los empleados.

Muchos de estos procesos ofrecen servicios, por ejemplo, networking (red), cups (servicio de impresión), ssh, {xdm, gdm, lighdm} (diferentes gestores de display), lvm (gestor de volúmenes lógicos), etc

Paralelismo: antiguamente se despertaba a los empleados uno a uno. `systemd` es moderno: despierta al de la luz, al del agua, al de la limpieza y al de seguridad **todos a la vez** para abrir más rápido

`systemctl` (el mando a distancia)

`systemd` controla servicios. Tú, como administrador, controlas a `systemd` usando el comando `systemctl`

Acción	Comando	Significado	Analogía Luz
Ahora mismo	<code>start</code> / <code>stop</code>	Enciende o apaga el servicio en este instante .	Pulsas el interruptor de la luz ahora.
Para el futuro	<code>enable</code> / <code>disable</code>	Configura si el servicio debe arrancar automáticamente la próxima vez que reinicies .	Programas el temporizador para que la luz se encienda sola mañana.

`systemctl status ssh` comprueba si hay servidor de ssh.

Si haces `systemctl stop ssh`, el servicio se para ahora. Pero si no haces `disable`, al reiniciar el PC, volverá a arrancar. También existe `systemctl restart ssh`

Los Targets (Los niveles de alerta)

`systemd` puede poner el ordenador en diferentes modos o estados de ánimo, llamados Targets.

Emergency / Rescue (Modo Pánico): El sistema está roto. Solo arranca lo mínimo vital. No hay gráficos, ni red, ni usuarios. Solo una pantalla negra con letras para que el administrador intente arreglarlo (en emergencia abre un único shell).

Multi-user (Modo Servidor): arranca todo **menos los gráficos**, es como se usan los servidores profesionales: ahorras memoria al no cargar ventanas ni ratón.

Graphical (Modo Normal): arranca todo lo anterior + la interfaz gráfica (ventanas, ratón, escritorio). Es como usas tu PC normalmente.

NOTA

Shell (cáscara, concha). Si el **kernel** (núcleo) es la parte blanda y vital de una nuez, el **Shell** es la cáscara dura que lo rodea.

Un **shell** es un programa informático que actúa como **intérprete de comandos**. Es el intermediario o traductor entre tú y el kernel.

Tu no hablas binario, y el kernel no habla español. Tu escribes `date` palabra humana. El **shell** lee eso, busca el programa correspondiente y le dice al kernel: ejecuta las instrucciones de tiempo del sistema. El kernel devuelve el resultado y el shell te lo muestra en pantalla.

Confusión Terminal vs Shell La **terminal** es la **ventana negra** que ves en la pantalla. Es solo un programa que muestra texto y envía lo que tecleas. El **shell** es el **software** que vive dentro de esa ventana, es quien procesa lo que escribes. Tu puedes abrir una ventana de terminal y cambiar el shell que usas dentro sin cerrar la ventana.

Igual que hay muchas marcas de coches, hay muchos tipos de Shells. En linux el más famoso es **Bash** pero hay otros:

- **Bash (Bourne Again Shell):** es el estándar. Robusto y universal
- **Sh (Bourne Shell):** el abuelo, muy antiguo y básico
- **Zsh (Z Shell):** el moderno, por defecto usado en MacOS. Tiene colores, autocompletado inteligente y muchas ayudas visuales. También existen modelos de terminales.

1.4 Verificación de la instalación

Linux soporta la inmensa mayoría del hardware actual (arquitecturas i386, AMD64, ARM, etc.). Durante la instalación, el sistema suele detectar y configurar los componentes automáticamente.

Para verificar esta detección y diagnosticar problemas, disponemos de herramientas que leen la información directamente del kernel y la BIOS.

1.4.1 Información de hardware

Comando	Descripción	¿Requiere Root?
<code>lscpu</code> / <code>nproc</code>	Información del procesador y número de núcleos.	No

Comando	Descripción	¿Requiere Root?
<code>lsram</code> / <code>free -h</code>	Estado de la memoria RAM y Swap.	No
<code>lspci</code>	Dispositivos conectados a la placa (Gráfica, Red, Sonido).	No
<code>lsusb</code>	Dispositivos externos USB (Teclado, Ratón, Pendrives).	No
<code>dmidecode</code>	Información profunda de la BIOS (Hardware físico real).	Sí (sudo)
<code>fdisk -l</code>	Muestra discos y particiones físicas.	Sí (sudo)
<code>df -h</code>	Muestra particiones montadas y espacio disponible.	No
<code>uname -a</code>	Versión del Kernel.	No
<code>lsb_release -a</code>	Versión de la Distribución (Debian, Ubuntu, etc.).	No

1.4.2 Información de Bajo Nivel (BIOS/DMI)

El comando `dmidecode` vuelca la información de la **DMI** (Desktop Management Interface). Es fundamental porque nos dice **qué hardware físico hay conectado**, incluso si el sistema operativo no tiene drivers para usarlo.

- **¿Qué muestra?** Modelo de placa base, slots de RAM ocupados/libres, versión de BIOS, números de serie, etc.
- **Uso común:** Saber cuánta RAM máxima soporta tu placa o qué modelo exacto de RAM tienes sin abrir el PC.

```
System Information
  Manufacturer: Gigabyte Technology Co., Ltd.
  Product Name: H81M-HD3
Memory Device
  Size: 4 GB
  Type: DDR3
  Speed: 1333 MT/s
```

1.4.3 Información general de hardware

El comando `lshw` muestra información general del hardware. La información es similar a la del comando anterior, si bien en este caso el comando lee la información de los directorios `/proc` y `/sys`

1.4.4 Información CPU

Comandos: `lscpu`, `nproc`

- **lscpu**: Muestra la arquitectura (x86_64), modo de operación (32/64 bits), velocidad (MHz), y el tamaño de las memorias Caché (L1, L2, L3), vitales para el rendimiento.
- **nproc**: Solo devuelve un número (el total de hilos de procesamiento disponibles).

1.4.5 Dispositivos y Buses

A. Bus PCI (**lspci**)

Muestra las tarjetas internas pinchadas en la placa base (o soldadas).

- **Ejemplos:** Tarjeta Gráfica (VGA/3D), Tarjeta de Red (Ethernet/WiFi), Tarjeta de Sonido.
- **Lectura:** `00:02.0 VGA compatible controller...` (Indica el slot y el dispositivo).

B. Bus USB (**lsusb**)

Muestra periféricos externos.

- **Ejemplos:** Teclados, ratones, webcams, discos externos.
- **Lectura:** `Bus 001 Device 005: ID 413c:1002...` (El ID es crucial para buscar drivers si algo no funciona).

1.4.6 Discos duros

Esta es una de las partes más críticas. Linux trata los discos como **archivos de dispositivo** en `/dev/`.

Nomenclatura de Discos

Tipo de Disco	Bus	Nombre en Linux	Ejemplo
SATA / SSD	Serial ATA	<code>sd</code> + letra (a, b, c...)	<code>/dev/sda</code> (Disco 1), <code>/dev/sdb</code> (Disco 2)
NVMe (Moderno)	PCI Express	<code>nvme</code> + controlador + dispositivo	<code>/dev/nvme0n1</code> (Disco NVMe 1)

Nomenclatura de Particiones

Se añade un número al final del disco.

- **SATA:** `/dev/sda1` (Partición 1 del disco A).
- **NVMe:** `/dev/nvme0n1p1` (Partición 1 del disco NVMe 0).

Comandos de Visualización

fdisk -l (La realidad física):

- Muestra **todo** lo que hay en el disco, esté en uso o no.

- Información: Tamaño de sectores, tipo de partición (Linux, Swap, EFI, Windows Data).

`df -h` (La realidad lógica):

- Muestra solo las particiones **montadas** (las que el sistema está usando ahora mismo).
- Información: Espacio total, usado y libre.
- *Nota:* Muestra sistemas de archivos virtuales como `tmpfs` (RAM usada como disco temporal).

Memoria (`free`)

Muestra el estado de la RAM y la Swap. **Opción clave:** Usar siempre `-h` (human readable) para ver Gigas (Gi) en vez de bytes.

```
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7,6Gi	1,0Gi	5,7Gi	19Mi	928Mi	6,4Gi
Swap:	52Gi	182Mi	51Gi			

Sistema y Kernel

Kernel (Motor)

- **Comando:** `uname -a`
- **Qué dice:** Versión del núcleo (ej. `6.8.0-45-generic`). Útil para saber si tu kernel soporta un hardware muy nuevo.

Distribución (Carrocería)

- **Comando:** `lsb_release -a`
- **Qué dice:** Versión comercial (ej. `Ubuntu 24.04 LTS`, `Debian 12`).

Módulos del Kernel (`*.ko`) Son los "drivers" de Linux. No son programas normales, son piezas de código que se insertan en el kernel vivo.

- `lsmod`: Lista lo que está cargado (ej. `nvidia`, `snd_hda_intel`).
- `modprobe [módulo]`: Carga un módulo (inteligente, carga dependencias).
- `rmmmod [módulo]`: Elimina un módulo (si no se está usando).

Directorios virtuales

Son directorios mágicos. **No están en el disco duro**, sino en la memoria RAM. Son la forma que tiene el Kernel de "hablar" contigo.

- Si haces `cat /proc/cpuinfo`, no estás leyendo un archivo de texto, estás pidiendo al Kernel que te "imprima" el estado actual de la CPU.

- La mayoría de comandos anteriores (`lscpu`, `lshw`, `free`) en realidad lo único que hacen es leer estos archivos, ordenarlos y mostrártelos bonitos.

2. Instalación de Software

2.1 Formas de instalación

Podemos clasificar la instalación en dos grandes familias según cómo nos llega el software:

Método	Descripción	Ventajas 	Inconvenientes 
Paquetería / Binarios (Estándar: <i>.deb, .rpm, .exe</i>)	El programa ya viene compilado ("cocinado") desde el servidor. Tú solo descargas el ejecutable listo para usar.	• Instalación rápida: No consume CPU procesando. • Gestión de dependencias: El gestor (APT/DNF) descarga las librerías necesarias automáticamente. • Actualizaciones sencillas: Un comando actualiza todo.	• Menor personalización: Viene con las opciones que decidió el creador, no tú. • Optimización genérica: Está hecho para funcionar en cualquier CPU, no exprime al 100% la tuya específica.
Código Fuente (Compilación manual)	Descargas el código "crudo" (texto legible por humanos) y tu ordenador lo traduce a lenguaje máquina (compilación).	• Máxima optimización: El programa se adapta a la arquitectura exacta de tu procesador. • Control total: Puedes activar o desactivar funciones específicas antes de "cocinarlo".	• Lento y Complejo: Tarda tiempo en compilar (CPU al 100%). • Dependencias manuales: A menudo fallará si te falta una librería. • Difícil de desinstalar: Si no usas herramientas extra, puede dejar "basura" en el sistema.

2.1.1 Gestores de paquetes

En la mayoría de distribuciones, usamos **Gestores de Paquetes**. Estos programas consultan una base de datos central (repositorios), descargan el software y lo instalan.

Ventajas del sistema de paquetes:

- **Centralización:** Todo el software viene de fuentes oficiales y verificadas (sin virus).
- **Dependencias:** Si instalas el "Programa A" y este necesita la "Librería B", el gestor descarga ambos automáticamente.
- **Mantenimiento:** Un solo comando actualiza *todo* el sistema y sus programas.

El mundo Linux se divide principalmente en dos "dialectos":

1. **Paquetes** `.deb` : Usados por **Debian**, Ubuntu, Linux Mint, Kali.
2. **Paquetes** `.rpm` : Usados por **RedHat**, Fedora, CentOS, SUSE.

2.1.3 Gestión de paquetes en Debian

Debian utiliza una estructura de herramientas por capas ("cebolla"). De más bajo nivel a más alto nivel:

Nivel bajo: `dpkg` (Debian Package)

Es la herramienta base. Gestiona los archivos `.deb` individuales que ya tienes descargados en tu disco.

- **Limitación grave: NO resuelve dependencias.** Si el paquete necesita una librería extra, `dpkg` fallará y te dirá que la busques tú.
- *Uso:* Instalar un `.deb` suelto descargado manualmente.

Nivel Alto: `apt` (Advanced Package Tool)

Es la herramienta inteligente que usamos a diario. Se conecta a internet (repositorios).

- **La magia:** Resuelve dependencias. Si pides instalar `vlc`, `apt` calculará qué librerías faltan, las descargará de los servidores y llamará a `dpkg` por detrás para instalarlas.
- *Fuentes:* Lee la lista de repositorios desde `/etc/apt/sources.list`.

Herramientas Especiales

- **Tiendas de Software (GUI):** Interfaces gráficas (como GNOME Software) que por debajo ejecutan `apt`.
- **alien** : Una herramienta de "traducción". Permite convertir un paquete `.rpm` (de RedHat) a `.deb` (para Debian).

2.1.4 La nueva Era: Paquetes Universales y Contenedores

Para solucionar el problema de "esto funciona en Fedora pero no en Ubuntu", nacieron los paquetes autocontenidos.

SNAP y **FLATPAK** Son paquetes que incluyen **todas sus dependencias dentro**. Llevan sus propias librerías y no usan las del sistema operativo

Característica	Paquete Tradicional (apt)	Paquete Universal (snap/flatpak)
Dependencias	Usa las librerías compartidas del sistema.	Autocontenido: Lleva sus propias librerías dentro.
Seguridad	Acceso total al sistema (generalmente).	Sandboxing: Se ejecutan en una "caja" aislada (contenedor).
Versiones	Depende de la versión de tu Linux.	Siempre la última versión disponible del desarrollador.
Tamaño	Pequeño (reusa librerías).	Grande (mucha redundancia de librerías).
Arranque	Rápido.	Lento (sobre todo el primer arranque).

2.1.5 Gestores de Lenguajes: PIP (Python)

A veces no instalamos un "programa", sino una librería para programar. Cada lenguaje tiene su propio gestor:

PIP es el estándar para instalar librerías del lenguaje Python:

- **pip** (o **pip2**): Para Python antiguo (versión 2.7, obsoleto).
- **pip3**: Para Python moderno (versión 3.x).

```
# Instalar el gestor pip
sudo apt install python3-pip

# Instalar una librería de Python (ej. NumPy para matemáticas)
pip3 install numpy

# Desinstalar
pip3 remove pytorch
```

2.2 **dpkg** - Debian Package

dpkg (Debian Package) es la herramienta base para gestionar paquetes **.deb**. A diferencia de **apt**, **dpkg** **no descarga nada de internet ni resuelve dependencias**. Solo trabaja con archivos que ya

tienes descargados localmente en tu disco duro.

2.2.1 Estructura de un Paquete DEB

Un archivo `.deb` es un contenedor comprimido (similar a un `.zip`) que incluye:

1. **Binarios:** El programa ejecutable, librerías y documentación.
2. **Metadatos:** Información de versión, dependencias, arquitectura y scripts de configuración (pre-instalación y post-instalación).

2.2.2 Nomenclatura estándar

Formato: `paquete-versión-build-arquitectura.deb`

Parte	Descripción	Ejemplo
Paquete	Nombre de la aplicación.	<code>ethereum</code>
Versión	Versión del desarrollador.	<code>0.10.11</code>
Build	Revisión/Compilación del empaquetador.	<code>1</code>
Arquitectura	CPU destino (amd64, i386, armhf).	<code>i386</code>
Extensión	Siempre <code>.deb</code>	<code>.deb</code>

2.2.3 Opciones `dpkg`

Acción	Opción Corta	Opción Larga	Argumento necesario
Instalar	<code>-i</code>	<code>--install</code>	Nombre del Fichero (.deb)
Eliminar	<code>-r</code>	<code>--remove</code>	Nombre del Paquete
Purgar	<code>-P</code>	<code>--purge</code>	Nombre del Paquete
Listar	<code>-l</code>	<code>--list</code>	(Opcional) Patrón
Estado	<code>-s</code>	<code>--status</code>	Nombre del Paquete
Ver Ficheros	<code>-L</code>	<code>--listfiles</code>	Nombre del Paquete
Buscar	<code>-S</code>	<code>--search</code>	Ruta de un fichero

2.2.4 Operaciones de instalación y eliminación

Instalación (`-i`)

Instala un paquete que tienes descargado.

- **⚠ Importante:** Es el único caso donde necesitas escribir el **nombre completo del archivo** (incluyendo `.deb`).
- **Dependencias:** Si el paquete necesita librerías que no tienes, `dpkg` fallará (no las descarga automáticamente).

```
# Sintaxis: dpkg -i [fichero.deb]
dpkg -i ethereal_0.10.11-1_i386.deb
```

Si falla por dependencias, un truco es ejecutar inmediatamente después `apt-get install -f` para que APT descargue lo que falta.

Eliminación (`-r` vs `-P`)

Aquí ya no usamos el nombre del archivo, sino el **nombre del paquete** (ej: `wget`).

1. **Eliminar (`remove`):** Desinstala el programa, pero **mantiene los ficheros de configuración** en `/etc`. Útil si planeas reinstalarlo en el futuro y no quieres perder tu configuración.

```
dpkg -r wget
```

2. **Purgar (`purge`):** Desinstala el programa y **borra también la configuración**. Limpieza total.

```
dpkg -P wget
```

Reconfiguración

Si un paquete ya instalado da problemas o quieres cambiar sus opciones iniciales (como el idioma o la distribución de teclado):

```
dpkg-reconfigure nombre_paquete
```

2.2.5 Consultas e información

Listar Paquetes (`-l`)

Muestra una tabla con todos los paquetes instalados. Es muy útil entender la primera columna (Estado).

```
dpkg -l "telnet*"
```

Interpretación de la salida (Las siglas `ii`, `un`, etc.): La primera columna tiene dos o tres letras.

- **1ª Letra (Deseado):** Lo que tú quieres que pase.
 - **i**: Install (Quieres que esté instalado).
 - **r**: Remove (Quieres desinstalarlo).
- **2ª Letra (Actual):** Lo que realmente hay.
 - **i**: Installed (Está instalado correctamente).
 - **c**: Config-files (Se borró el programa, pero quedan archivos de configuración).
 - **n**: Not-installed (No está instalado).

El objetivo: Un paquete sano debe mostrar **ii** (Deseado: Instalar / Actual: Instalado).

Ver Estado Detallado (**-s**)

Muestra toda la información técnica (versión, dependencias, tamaño).

```
$ dpkg --status wget
Package: wget
Status: install ok installed    <-- ESTADO IMPORTANTE
Priority: important
Section: web
Version: 1.10-2
Depends: libc6, libssl0.9.7    <-- DEPENDENCIAS
Description: Retrieves files from the web...
```

Ver Ficheros de un Paquete (**-L**)

¿Qué ha instalado este paquete y dónde?

```
$ dpkg -L wget
/usr/bin/wget          (El ejecutable)
/etc/wgetrc            (La configuración)
/usr/share/man/man1/.. (El manual)
```

Búsqueda Inversa (**-S**)

¿Tienes un archivo misterioso y no sabes qué paquete lo puso ahí?

```
# ¿A quién pertenece este fichero?
$ dpkg -S /usr/bin/wget
wget: /usr/bin/wget
```


2.3 APT - Advanced Packaging Tools

APT es la herramienta de gestión de paquetes de alto nivel del proyecto Debian. Mientras que `dpkg` trabaja con archivos locales, **APT** gestiona todo el ciclo de vida del software conectándose a repositorios de internet.

Sus superpoderes son:

1. **Resolución de Dependencias:** Si quieres instalar el *Programa A*, y este necesita la *Librería B*, APT descarga e instala ambos automáticamente.
2. **Repositorios Remotos:** Descarga el software desde servidores oficiales (mirrors).
3. **Actualización Integral:** Actualiza todo el sistema operativo con un solo comando.

2.3.1 Archivo de configuración

La lista de fuentes (`sources.list`)

El archivo `/etc/apt/sources.list` es el mapa del tesoro. Le dice a APT **dónde** buscar los paquetes.

Sintaxis de una línea: `[Tipo] [URI] [Distribución] [Componentes]`

Ejemplo real desglosado:

```
deb http://ftp.es.debian.org/debian/ bookworm main contrib non-free
```

Campo	Significado	Opciones comunes
Tipo	¿Qué bajamos?	<code>deb</code> (Binarios/Programas listos)
		<code>deb-src</code> (Código fuente para compilar)
URI	¿Dónde está?	<code>http://...</code> , <code>ftp://...</code> , <code>file://...</code>
Distribución	¿Qué versión?	<code>stable</code> (Estable/Recomendada)
		<code>testing</code> (Pruebas/Más nueva)
		<code>unstable</code> / <code>sid</code> (Desarrollo/Inestable)
Componentes	¿Qué licencia?	<code>main</code> (Software libre oficial)

Campo	Significado	Opciones comunes
		<code>contrib</code> (Libre pero depende de no-libres)
		<code>non-free</code> (Software privativo/propietario)

Configuración Avanzada (`apt.conf`)

Para cambiar el comportamiento de APT (ej. configurar un proxy), se usan estos ficheros:

1. `/etc/apt/apt.conf` : Fichero único (estilo clásico).
2. `/etc/apt/apt.conf.d/` : Directorio modular (estilo moderno). Aquí se crean ficheros pequeños (ej: `01proxy`, `99mysettings`). APT los lee todos en orden numérico. Es más ordenado y fácil de administrar.

2.3.2 Comandos y opciones de APT

Históricamente había dos comandos separados: uno para *hacer* cosas (`apt-get`) y otro para *buscar* cosas (`apt-cache`). Modernamente, se recomienda usar `apt`, que unifica a los dos anteriores y añade barras de progreso y colores.

Acción	Comando Moderno (Recomendado)	Comando Clásico (Scripts)	Comando Consulta
Actualizar lista	<code>apt update</code>	<code>apt-get update</code>	-
Instalar	<code>apt install</code>	<code>apt-get install</code>	-
Eliminar	<code>apt remove</code>	<code>apt-get remove</code>	-
Buscar	<code>apt search</code>	-	<code>apt-cache search</code>
Mostrar info	<code>apt show</code>	-	<code>apt-cache show</code>

2.3.3 Flujo de Trabajo (Operaciones comunes)

Actualizar el catálogo

¡Vital! Antes de instalar nada, hay que sincronizar el índice local con el servidor remoto.

```
apt update
```

Actualizar el sistema

Actualiza los programas instalados a su última versión disponible

```
apt upgrade
```

- **apt upgrade**: Actualiza paquetes de forma segura. No borra paquetes existentes.
- **apt dist-upgrade** (o **full-upgrade**): Es más agresivo. Si para actualizar el Navegador X necesita borrar la Librería Y, lo hará. Es necesario para cambios grandes de versión de Debian.

Instalar paquetes

```
apt install firefox
```

Eliminar paquetes

Hay dos formas de borrar:

1. **remove**: Desinstala el programa pero **deja los archivos de configuración** (útil si crees que volverás a instalarlo).

```
apt remove firefox
```

2. **purge**: Desinstala el programa y **borra todo rastro** de configuración.

```
apt purge firefox
```

Limpieza

APT guarda los **.deb** descargados en **/var/cache/apt/archives/**. Con el tiempo, esto ocupa mucho espacio.

- **apt autoremove**: Borra dependencias "huérfanas" (librerías que se instalaron para un programa que ya has borrado y que nadie más usa).
- **apt clean**: Borra los instaladores **.deb** descargados de la caché para liberar disco.

2.3.4 Gestión de Fuentes

Solo para usuarios avanzados que necesitan modificar el código. Requiere líneas **deb-src** en el **sources.list**

- **apt-get source paquete**: Descarga el código fuente.

- `apt-get build-dep paquete`: Instala automáticamente todas las herramientas y librerías necesarias para compilar ese paquete.
- `apt-get source --compile paquete`: Descarga y compila directamente creando un `.deb`

2.3.5 Comando apt-cache

Mientras que `apt-get` se encarga de *instalar* y *modificar*, `apt-cache` es la herramienta de **consulta e investigación**.

Su función es leer la base de datos local (la caché que descargaste con `apt update`) para darte información sobre los paquetes disponibles, **incluso si no los tienes instalados**.

Buscar Paquetes (`search`)

Funciona como un buscador (Google) dentro de tu lista de software. Busca el texto que le indiques tanto en el **nombre** como en la **descripción** del paquete.

- **Sintaxis:** `apt-cache search [patrón]`
- **Detalle:** Admite expresiones regulares (Regex) para búsquedas avanzadas.

```
$ apt-cache search firefox
# Salida de ejemplo:
firefox - Mozilla Firefox web browser
firefox-locale-es - Spanish language pack for Firefox
iceweasel - Web browser based on Firefox (Transitional package)
```

Ficha Técnica del Paquete (`show`)

Muestra toda la información detallada ("metadatos") de un paquete específico.

- **Uso:** Leer la descripción larga, ver quién es el mantenedor, el tamaño de la descarga o la suma de verificación (SHA256).

```
apt-cache show nombre_paquete
```

Ver Dependencias (`depends`)

Muestra el árbol de relaciones del paquete. Es útil para saber **qué más se instalará** si decides instalar ese paquete.

- **Depends:** Lo que es obligatorio instalar.
- **Suggests:** Lo que te sugiere instalar (opcional).
- **Conflicts:** Con qué programas es incompatible.

```
$ apt-cache depends nano
nano
  Depends: libc6
  Depends: libncursesw6
  Depends: libtinfo6
  Suggests: spell
```

Política de Versiones (`policy`)

Este es el comando más técnico y útil para diagnosticar problemas. Te dice **de dónde** viene un paquete y **qué versión** tiene prioridad.

Muestra tres datos clave:

1. **Installed:** La versión que tienes instalada ahora (o `(none)` si no lo tienes).
2. **Candidate:** La versión que se instalaría si ejecutaras `install` ahora mismo.
3. **Version Table:** La lista de todos los repositorios que ofrecen este paquete y su prioridad

```
$ apt-cache policy firefox
firefox:
  Installed: (none)
  Candidate: 115.0-2
  Version Table:
    115.0-2 500
    500 http://deb.debian.org/debian stable/main amd64 Packages
```

2.3.6 Solución de Problemas

Dependencias Rotas

Si una instalación falla a medias o se va la luz, el sistema de paquetes puede quedar "roto".

```
# El comando mágico para arreglarlo
apt --fix-broken install
# (o su versión antigua: apt-get -f install)
```

Esto intenta descargar lo que falta y configurar lo que quedó a medias.

El fichero de bloqueo (Lock File)

APT usa un archivo "cerrojo" en `/var/lib/dpkg/lock` para asegurar que **solo una persona instala cosas a la vez**.

Síntoma: Intentas usar apt y dice: `E: Could not get lock /var/lib/dpkg/lock - open (11: Resource temporarily unavailable)`

Causas:

1. Tienes otra terminal abierta instalando algo.
2. El sistema de actualizaciones automáticas (`unattended-upgrades`) está trabajando en segundo plano.
3. Se fue la luz durante una instalación y el fichero no se borró.

Solución:

1. Esperar (lo más recomendable).
2. Verificar si hay procesos apt corriendo: `ps aux | grep apt`.
3. (Último recurso) Borrar el fichero manualmente si estás 100% seguro de que nada está corriendo

2.4 Instalación desde el código fuente

Este método consiste en descargar el código "crudo" creado por el programador y transformarlo (compilarlo) en un programa ejecutable adaptado específicamente a tu ordenador.

2.4.1 Flujo de trabajo estándar

Descarga del código (Tarballs)

El código suele venir empaquetado y comprimido para ocupar menos espacio. Las extensiones nos dicen cómo tratarlo:

- **.tar:** Empaquetado (agrupado en un solo fichero) pero sin comprimir.
- **.tar.gz / .tgz:** Empaquetado y comprimido con Gzip (el más común).
- **.tar.bz2 / .tbz:** Empaquetado y comprimido con Bzip2 (comprime más, tarda más).

Desempaquetado

Usamos el comando `tar` (Tape ARchiver).

- Descomprimir .tar.gz: `tar -xzvf archivo.tar.gz`
- Descomprimir .tar.bz2: `tar -xjvf archivo.tar.bz2`

Nota: Tras descomprimir, siempre se crea un directorio nuevo. Debemos entrar en él: `cd nombre-directorio`.

Leer la instrucciones (`README` / `INSTALL`)

Dentro del directorio siempre hay ficheros de texto con instrucciones vitales. **Léelos antes de seguir.** Te dirán qué dependencias necesitas instalar antes de empezar.

Configuración (`.configure`)

Ejecutamos el script `./configure` que viene con el código (generado por herramientas como `autoconf`).

- **¿Qué hace?** Chequea tu sistema. Verifica si tienes el compilador (gcc), si te faltan librerías y si tu entorno es compatible.
- **Resultado:** Si todo va bien, crea un fichero llamado `Makefile`. Este fichero es la "receta" personalizada para tu PC.
- **Personalización:** Puedes decirle dónde instalar el programa usando la opción `--prefix`.

```
./configure --prefix=/home/usuario/mi_programa
```

Compilación (`make`)

El comando `make` lee el `Makefile` generado en el paso anterior y empieza a compilar.

- `make`: Compila el código (traduce C/C++ a binario). Tarda desde segundos hasta horas.
- `make clean`: Si la compilación falla o quieres empezar de cero, esto borra todos los archivos temporales creados.

Instalación (`make install`)

Copia los binarios finales y las librerías a sus destinos definitivos (normalmente `/usr/bin`, `/usr/lib`, etc.).

- **Permisos:** Si instalas en directorios del sistema, necesitas `sudo make install`. Si usaste `--prefix` a tu carpeta, no hace falta.

2.4.2 Tipos de Ejecutables y Librerías

Una vez compilado el programa, obtenemos un **binario**. Dependiendo de cómo se haya "enlazado" (linked), puede ser de dos tipos:

Tipo	Descripción	Ventajas/Desventajas
Estático (Static)	El ejecutable lleva todas las librerías que necesita metidas dentro de sí mismo. Es un bloque único.	✓ Funciona en cualquier sistema. ✗ Ocupa mucho espacio en disco y

Tipo	Descripción	Ventajas/Desventajas
		RAM. ❌ Difícil de actualizar (hay que recompilar todo).
Dinámico (Dynamic)	El ejecutable es pequeño. Cuando se inicia, pide al sistema operativo las librerías que necesita (<code>.so</code>).	✅ Ocupa muy poco espacio. ✅ Si actualizas una librería del sistema, todos los programas mejoran. ❌ Si falta una librería, el programa no arranca.

2.4.3 Gestión de Librerías Dinámicas (`ldd`)

Como los programas dinámicos dependen de archivos externos, a veces fallan porque no los encuentran. Para diagnosticar esto usamos `ldd` (List Dynamic Dependencies).

Verificar dependencias

El comando muestra qué librerías pide el programa y dónde las ha encontrado el sistema.

Formato: `librería_necesaria => librería_encontrada (dirección_memoria)`

```
$ ldd /bin/ls
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000...)
    libpcre.so.3 => /lib/x86_64-linux-gnu/libpcre.so.3 (0x0000...)
    ...

### Problemas comunes ("Not found")
Si ves esto, el programa no arrancará:
```shell
libgdal.so.20 => Not found
```

#### Posibles causas y soluciones:

1. **La librería no está instalada:** Tienes que instalar el paquete que la contiene (usando `apt` ).



2. **La librería está en una ruta rara:** El sistema busca en `/lib` y `/usr/lib`. Si tu librería está en `/opt/mi_app/lib`, el sistema no la ve.

- *Solución:* Añadir esa ruta a la variable de entorno `LD_LIBRARY_PATH`.

- *Ejemplo:* `export LD_LIBRARY_PATH=/opt/mi_app/lib:$LD_LIBRARY_PATH`

3. **Error de Versión (Version Mismatch):** El programa pide `lib.so.3` y tú tienes `lib.so.4`.

- *Solución "sucia" (Symlink):* Engañar al programa creando un enlace simbólico.

```
Hacemos creer al sistema que la versión 4 es la 3
ln -s libgdal.so.30.0.4 libgdal.so.30.0.3
```

## El cargador dinámico ( `ld.so` )

Es el programa "invisible" del sistema operativo que se ejecuta cada vez que abres una aplicación. Su trabajo es leer la lista de dependencias del ejecutable, buscar esos archivos `.so` en el disco duro y cargarlos en la RAM para que el programa funcione.

## 3. Automatización de Tareas

---

En Linux, diferenciar el "cuándo" se ejecuta una tarea es clave. Tenemos dos herramientas principales según nuestras necesidades:

1. **at** / **batch**: Para tareas que se ejecutan **una sola vez** en el futuro (ej: "Apaga el servidor a las 3 de la mañana").
2. **cron**: Para tareas **repetitivas** (ej: "Haz una copia de seguridad todos los viernes").

### 3.1 Ejecución Diferida (Una sola vez): **at** y **batch**

Estas herramientas ponen tareas en una cola de espera.

#### Comando **at** (A una hora fija)

---

Programa un comando para ejecutarse en un momento específico.

**Sintaxis:** **at** [hora/fecha]

**Flujo de trabajo interactivo:**

- Escribes **at 11:45**
- El sistema te da un prompt (**>at**) para que escribas los comandos que quieras
- Pulsas **CTRL+D** para guardar y salir (EOT)

Si el comando genera texto (ej: un **ls**), el sistema te enviará un **email** interno con el resultado (a menos que redirijas la salida **> archivo**). Es **persistente**, se guarda en disco; si reinicias, el trabajo sigue pendiente (si la hora no ha pasado).

```
$ at 11:45
warning: commands will be executed using /bin/sh
at> ls /tmp > /home/usuario/lista_tmp.txt # Guarda el listado en un fichero
at> env DISPLAY=:0 zenity --info --text="Hola" # Muestra ventana gráfica (requiere
variables de entorno)
at> <CTRL-D>
job 4 at Wed Nov 16 11:45:00 2025
```

#### Comando **batch** (Cuando el sistema descansa)

---

Es una variante de **at**. No le dices "a qué hora", sino "hazlo cuando puedas"

- Ejecuta el trabajo solo cuando la **carga del sistema** baja de 1.5
- Ideal para tareas pesadas que no quieres que ralenticen el ordenador mientras lo usas

## Gestión de la cola (`atq` y `atrm`)

- `atq` (**Queue**): Lista los trabajos pendientes.
  - El usuario normal ve los suyos. El root ve los de todos.
- `atrm [ID]`: Borra un trabajo específico (usando el número ID que da `atq`).

## 3.2 Ejecución Periódica (Recurrente): `cron`

**Cron** es el reloj del sistema. Utiliza un demonio (servicio en segundo plano) que comprueba cada minuto si hay algo que hacer.

### Gestión del fichero: `crontab`

Cada usuario tiene su propia tabla de tareas. NO se edita el fichero a mano, se usa el comando:

Comando	Acción
<code>crontab -e</code>	<b>Editar</b> tu tabla de tareas (abre nano/vim).
<code>crontab -l</code>	<b>Listar</b> tus tareas programadas.
<code>crontab -r</code>	<b>Remove</b> (borrar) toda tu tabla de tareas.

## Sintaxis de una línea de Cron

Cada línea en el fichero represent una tarea y tiene **6 campos**: 5 de tiempo y 1 de comando.

**Estructura:** `minuto hora día_mes mes día_semana comando`

Campo	Valores permitidos	Notas
<b>Minuto</b>	<code>0-59</code>	
<b>Hora</b>	<code>0-23</code>	Formato 24h.
<b>Día del mes</b>	<code>1-31</code>	
<b>Mes</b>	<code>1-12</code>	(O nombres: jan, feb...)
<b>Día Semana</b>	<code>0-7</code>	<b>0 y 7 son Domingo.</b> 1=Lunes, etc.
<b>Comando</b>	Ruta comando	Lo que se ejecutará.

## Operadores Especiales

- `*` (Asterisco): **"Todos"**. (Ej: `*` en horas significa "todas las horas").
- `,` (Coma): **Listas**. (Ej: `1,15` = min 1 y min 15).
- `-` (Guion): **Rangos**. (Ej: `1-5` = de Lunes a Viernes).
- `/` (Barra): **Pasos/Repetición**. (Ej: `*/15` = Cada 15 unidades).
  - `*/2` en horas = A las 0, 2, 4, 6... (cada 2 horas).

## Variables de Entorno en Cron

Al principio del fichero puedes definir variables globales:

- `MAILTO=pepe`: Si el comando falla o dice algo, envía el email a "pepe" (si está vacío `MAILTO=""`, descarta los emails).
- `SHELL=/bin/bash`: Qué intérprete usar.

## Ejemplo

```
Borra temporales de lunes a viernes (1-5) a las 04:30 AM
30 4 * * 1-5 rm -rf /tmp/*

Ejecuta un script cada 15 minutos (0, 15, 30, 45)
Rango horario: de 00:00 a 08:00 y de 20:00 a 23:00
*/15 0-8,20-23 * * * /home/usuario/script_noche.sh

Ejecuta algo el día 1 de cada mes a las 00:00
0 0 1 * * /root/backup_mensual.sh
```

## 3.3 Cron del Sistema (Directorios)

El administrador del sistema (root) tiene una forma alternativa de programar tareas sin pelearse con la sintaxis de los asteriscos.

Existen directorios especiales en `/etc/`. Si metes un script ejecutable dentro, el sistema lo lanzará automáticamente según el nombre de la carpeta:

- `/etc/cron.hourly`: Se ejecuta cada hora (en el minuto 17 aprox).
- `/etc/cron.daily`: Se ejecuta una vez al día (normalmente a las 6:25 AM).
- `/etc/cron.weekly`: Una vez a la semana.
- `/etc/cron.monthly`: Una vez al mes.

## 4. Copias de Seguridad

El trabajo del administrador no es evitar que los desastres ocurran (eso es imposible), sino asegurar que **se puede recuperar la información** cuando ocurran.

### Causas de pérdida de datos:

- Error humano (borrado accidental).
- Fallos técnicos (disco duro roto, corrupción de software).
- Catástrofes (incendio, inundación, robo).
- Malware (Ransomware, ataques hackers).

### 4.1 Los Tres Pilares del Backup

Para montar un sistema de copias necesitamos definir tres cosas: **¿Dónde?, ¿Cómo? y ¿Cuándo?**

#### Medios de almacenamiento (¿Dónde?)

Medio	Capacidad	Uso ideal
<b>Cintas (LTO)</b>	Muy alta (18TB+ en LTO-9)	Estándar empresarial. Son baratas por TB y duraderas.
<b>Librerías Robotizadas</b>	Exabytes	Grandes Centros de Datos. Un robot mueve las cintas automáticamente.
<b>Discos Duros (HDD)</b>	Media/Alta (TB)	Pequeñas empresas o copias rápidas locales.
<b>Discos Ópticos</b>	Baja (DVD/BluRay)	Casi en desuso, salvo para archivo a muy largo plazo (Cold Storage).
<b>Nube (Cloud)</b>	"Ilimitada"	Copias externas (off-site). <i>Ojo: Depende de tu velocidad de internet y de la legislación de protección de datos.</i>

#### El mecanismo de Copia (¿Cómo?)

Existen dos filosofías técnicas para copiar los datos:

##### Copia Fichero a Fichero (Nivel Lógico):

- El programa pide al Sistema Operativo: "Dame el archivo `foto.jpg`", luego "Dame `texto.txt`".
- **Ventaja:** Puedes recuperar un solo archivo fácilmente.
- **Desventaja:** Lento si hay millones de archivos pequeños.

- *Herramienta:* `tar`.

### Copia de Imagen (Nivel físico/bloque):

- El programa no mira archivos, mira **bits**. Copia el disco duro sector por sector (clona los ceros y unos).
- **Ventaja:** Extremadamente rápido y exacto (copia hasta lo borrado si quieres).
- **Desventaja:** Restaurar un solo archivo es un dolor de cabeza (tienes que restaurar la imagen entera).
- *Herramientas:* `dd`, `dump`.

## El planificador (¿Cuándo?)

---

Define la política de copias. Herramientas como `cron` se encargan de ejecutar la copia a las 3:00 AM, por ejemplo.

## 4.2 Estrategias: Completo, Diferencial e Incremental

### Backup Completo (Nivel 0)

---

- **Qué hace:** Copia **TODO**. Da igual si se modificó o no.
- **Ventaja:** Para restaurar solo necesitas esta cinta.
- **Desventaja:** Tarda muchísimo y ocupa mucho espacio.

### Backup Diferencial

---

- **Qué hace:** Copia todo lo que ha cambiado **desde el último Backup Completo (Nivel 0)**.
- **Analogía:** Imagina que el Domingo haces una foto a toda tu casa (Completo).
  - *Lunes:* Pintas una pared de rojo. El backup guarda "Pared roja".
  - *Martes:* Compras un sofá. El backup guarda "Pared roja + Sofá".
- **Restauración:** Rápida. Necesitas el **Completo + El último Diferencial**. (2 pasos).
- **Espacio:** Crece día a día (el viernes copias todo lo acumulado en la semana).

### Backup Incremental

---

- **Qué hace:** Copia solo lo que ha cambiado **desde el último backup (sea del tipo que sea)**.
- **Analogía:**
  - *Domingo:* Foto casa (Completo).
  - *Lunes:* Pintas pared roja. Backup guarda solo "Pared roja".
  - *Martes:* Compras sofá. Backup guarda solo "Sofá". (No guarda la pared, porque eso ya se guardó el lunes).
- **Restauración:** Lenta. Necesitas el **Completo + Lunes + Martes + Miércoles...** (Muchos pasos).

- **Espacio:** Es el que menos ocupa.

## 4.3 Niveles de Dump y Estrategias de Rotación

El comando `dump` utiliza números (Niveles 0-9) para definir qué copiar. **Regla de Oro:** Un nivel `N` copia todo lo modificado desde el último backup de nivel *inferior* a `N`.

### Ejemplo 1: Estrategia Diferencial

---

Secuencia: `0, 5, 5, 5, 5` (Domingo a Jueves)

1. **Domingo (Nivel 0):** Copia TODO.
  2. **Lunes (Nivel 5):** Busca el último nivel menor que 5. Encuentra el 0 (Domingo). Copia cambios Lunes vs Domingo.
  3. **Martes (Nivel 5):** Busca el último nivel menor que 5. Encuentra el 0 (Domingo). Copia cambios (Lunes+Martes) vs Domingo.
- **Resultado:** Cada día copias más datos ("acumulativo").
  - **Recuperación:** Si se rompe el disco el Jueves, solo necesitas la cinta del **Domingo (0)** y la del **Miércoles (5)**.

### Ejemplo 2: Estrategia Incremental

---

Secuencia: `0, 3, 4, 5, 6` (Escalera de números)

1. **Domingo (Nivel 0):** Copia TODO.
  2. **Lunes (Nivel 3):** Busca menor que 3 -> Encuentra 0. Copia cambios del Lunes.
  3. **Martes (Nivel 4):** Busca menor que 4 -> Encuentra 3 (Lunes). Copia cambios Martes vs Lunes.
  4. **Miércoles (Nivel 5):** Busca menor que 5 -> Encuentra 4 (Martes). Copia cambios Miércoles vs Martes.
- **Resultado:** Cada día copias poquísimo (solo lo de ese día).
  - **Recuperación:** Si se rompe el disco el Jueves, necesitas: **Domingo (0) + Lunes (3) + Martes (4) + Miércoles (5)**. ¡Si pierdes una cinta intermedia, pierdes los datos siguientes!

## 4.4 Herramientas de Comando

`dump` y `restore` (Específicos de sistemas de ficheros)

---

Están diseñados para trabajar a bajo nivel con sistemas de ficheros ext2/ext3/ext4.

- **Ventaja:** Respetan permisos, fechas y saben si un archivo se movió. Soportan los "niveles" nativamente.
- **Fichero clave:** `/var/lib/dumpdates` (Aquí anota cuándo hizo el último backup para calcular los incrementales).

`dump`, su función es volcar los datos del sistema de ficheros a un destino (cinta, archivo o disco remoto). `dump [-nivel] [opciones] [ficheros_a_salvar]`

```
1. Backup COMPLETO (Nivel 0) de la partición /home en una cinta
Es importante usar el dispositivo (/dev/sda2) o el punto de montaje.
dump -0u -f /dev/st0 /home

2. Backup INCREMENTAL (Nivel 1)
Solo guarda lo modificado desde el último Nivel 0.
dump -1u -f /dev/st0 /home

3. Backup a un archivo normal (si no tienes cintas)
dump -0u -f /mnt/backup_disco_externo/home_full.dump /home
```

`restore`, permite recuperar desde un fichero completo o restaurar archivos sueltos. **Importante:** `restore` siempre extrae los datos **en el directorio donde te encuentres situado** en ese momento. `restore acción [opciones] [ficheros_a_salvar]`.

```
1. Vamos al sitio donde queremos dejar el fichero recuperado (seguridad)
cd /tmp

2. Abrimos el backup en modo interactivo
restore -if /dev/st0

Ahora estás dentro de una consola propia de restore (>).
Comandos dentro de restore:
> ls (listar ficheros del backup)
> cd usuario (entrar en carpetas)
> add informe.pdf (marcar el fichero que quieres recuperar)
> extract (ejecutar la extracción)
> quit (salir)
```

```
1. Es OBLIGATORIO ir al directorio raíz del nuevo disco
cd /mnt/nuevo_home

2. Restaurar todo el contenido del backup nivel 0
restore -rf /dev/st0
```



```
(Si tuvieras incrementales nivel 1, 2... tendrías que restaurarlos
después en orden secuencial).
```

## **tar** (Tape ARchiver - El estándar)

Empaqueta archivos. Es el más usado para backups de carpetas específicas.

- **Nota:** No tiene "niveles" automáticos como dump, hay que programarlos con scripts o usar la opción `--listed-incremental`.

```
Crear un backup comprimido
tar -czvf backup_home.tar.gz /home
```

## **dd** (Data Definition - Clonado en crudo)

Copia bit a bit.

- **Uso:** Clonar discos enteros, crear imágenes ISO, o copias forenses.
- **Peligro:** Si te equivocas con el destino (`of=`), sobrescribes el disco sin preguntar.

```
Clonar partición sda1 a sda2
dd if=/dev/sda1 of=/dev/sda2 bs=4M status=progress

if = Input File (Origen)
of = Output File (Destino)
bs = Block Size (Tamaño del bloque, para ir más rápido)
```

## 5. Programación de Scripts

### NOTA

Este tema originalmente habla también del shell y programar con bash, yo no explico porque no entra en el examen. Aunque es bastante importante saber como usar cada herramienta. En SOI se suele explicar (pero de forma muy precaria), si os interesa los apuntes originales de la asignatura lo explica bastante bien. Los temas tratados son:

- Línea de comandos: El interprete de comandos (shell), variables de shell, expansiones del shell, redirección de la entrada/salida, orden de evaluación y ficheros de inicialización de bash
- Scripts de administración: programación shell-script, entrada/salida, tests, estructura if...then...else, expresiones, control de flujo y funciones

De todas formas, yo seguramente hable de algunos temas recogidos aquí con tal de aclarar cosa que considero poco obvias.

### 5.1 Expresiones regulares

Las **expresiones regulares** permiten reconocer en texto una serie de cadenas de caracteres que siguen un patrón:

- Los ficheros de configuración y logs de Unix son normalmente ficheros de texto.
- Muchos comandos de procesamiento y búsqueda de texto como `grep`, `egrep`, `sed`, `awk` o `vi` usan expresiones regulares

No hay que confundir las expresiones regulares con los comodines, que son utilizados por el shell para referenciar ficheros. Se pueden usar con ficheros o con cualquier otro de tipo de entrada, como la entrada estándar, pipes, etc. Para evitar confusiones conviene escribir las expresiones regulares entre comillas, simples o dobles.

Se distinguen las expresiones regulares básicas y las extendidas, más completas.

## INFO

- **Los comodines (Wildcards/Globbing):** Son caracteres especiales (como `*` o `?`) interpretados por la **Shell** (antes de ejecutar el comando) para coincidir con **nombres de ficheros y directorios**.
- **Las expresiones regulares (Regex):** Son patrones interpretados por el **comando o programa** (como `grep`) para buscar coincidencias dentro del **contenido del texto**.
- **La entrada estándar (stdin):** Es el flujo de datos de entrada predeterminado de un programa (Descriptor de fichero 0). Si no se especifica un archivo, comandos como `grep` leerán lo que escribas en el teclado o lo que le llegue desde otro comando.
- **Los pipes (tuberías `|`):** Son el mecanismo que conecta la salida estándar (stdout) de un comando con la entrada estándar (stdin) del siguiente, permitiendo filtrar con expresiones regulares datos generados "al vuelo".

### 5.1.1 Comandos `egrep` y `sed -r`

Estos comandos permiten utilizar **Expresiones Regulares Extendidas (ERE)**, lo que simplifica la escritura de patrones complejos (como grupos `()` o alternancias `|`) sin necesidad de "escapar" tantos caracteres con la barra invertida `\`.

**El comando `egrep`:** busca parones en ficheros de texto.

- Lee el texto línea por línea. Si encuentra el patrón en una línea, imprime la línea completa en la salida estándar (pantalla). Si no, la descarta.
- **Diferencia clave:** `egrep` es equivalente a ejecutar `grep -E`. Interpreta los metacaracteres extendidos (`?`, `+`, `{}`, `|`, `()`) directamente.

```
egrep [opciones] 'patrón' archivo
```

**El comando `sed -r`:** Es un editor de flujo de texto. Aunque tiene muchas funciones, su uso más frecuente con expresiones regulares es para **sustituir** texto.

- **Opción `-r`:** Habilita las expresiones regulares extendidas (en versiones modernas se usa `-E`, pero `-r` es el clásico en GNU/Linux).
- **Sintaxis de sustitución:** `s` (substituye), `patrón` (lo que busco), `reemplazo` (lo nuevo) y `g` (global - todas las veces que aparezca en la línea).

```
sed -r 's/patrón/sustitución/g' archivo
```

## \*\*CARACTERES ESPECIALES EN EL REEMPLAZO (`&` Y `\1`):\*\*

Cuando usamos expresiones regulares en `sed`, la parte del "reemplazo" permite reciclar texto que hemos encontrado en la parte del "patrón":

1. **Los paréntesis** `(...)`: En el patrón de búsqueda `(.*)\.`foo, los paréntesis sirven para **capturar** y guardar en memoria una parte del texto (en este caso, todo lo que haya antes del punto).
2. **\1 (Retroreferencia)**: Recupera el contenido guardado en el **primer** par de paréntesis.
  - Si tuviéramos más paréntesis, usaríamos `\2`, `\3`, etc.
  - En el ejemplo: `\1` contiene solo el nombre del archivo (sin la extensión).
3. **& (Coincidencia completa)**: Es una variable especial que contiene **todo el texto** que coincidió con el patrón de búsqueda completo.
  - En el ejemplo: `&` contiene el nombre del archivo original completo (nombre + extensión).

Ambos comandos siguen la filosofía Unix:

- **Entrada:** Pueden leer de un fichero o de la **entrada estándar (stdin)** a través de una tubería (`|`).
- **Uso de Comillas:** Es vital proteger la expresión regular para que la Shell no la interprete antes de tiempo.

Tipo de Comillas	Descripción	Ejemplo
<b>Simples</b> '...'	<b>Recomendado.</b> Todo lo de adentro es literal. La shell no toca nada.	<code>grep '\$USER'</code> (Busca la palabra literal "\$USER")
<b>Dobles</b> "..."	Permite a la shell expandir variables antes de ejecutar el comando.	<code>grep "\$USER"</code> (Busca tu nombre de usuario actual, ej: "juan")

### Ejemplos:

```
Busca la palabra error o la palabra warning
egrep 'error|warning' /var/log/syslog
```

```
Cambia '2023' por '2024' en todas las apariciones
sed -r 's/2023/2024/g' config.txt
```

```
IP="192.168.1.1"
```

```
INCORRECTO: Busca la palabra literal $IP
```

```
egrep '$IP' conexiones.log
```

```
CORRECTO: La shell cambia $IP por 192.168.1.1 antes de buscar
egrep "$IP" conexiones.log
```

**Importante:** no debemos confundir las expresiones regulares con los comodines para la sustitución de nombres de ficheros (glob). Por ejemplo, el asterisco `*` como comodín indica cualquier cadena en nombres de fichero, mientras que como expresión regular es repetición.

Característica	Comodines (Globbing)	Expresiones Regulares (Regex)
Quién lo interpreta	La Shell (bash, zsh, sh)	La herramienta (grep, sed, awk, vim)
Objetivo	Encontrar archivos en el disco	Encontrar texto dentro de archivos/flujos
Ejemplo <code>*</code>	<code>*.txt</code> (Todos los archivos acabados en .txt)	<code>a*</code> (La letra 'a' cero o más veces)
Cuándo ocurre	Antes de que el comando se ejecute	Mientras el comando se ejecuta

Si no especificamos fichero, `egrep` y `sed` usan la entrada estándar. Las comillas dobles permiten utilizar variables y comandos dentro de las expresiones regulares.

## 5.1.2 Expresiones regulares extendidas

Los sistemas UNIX actuales admiten extensiones a las expresiones regulares básicas (BRE). Para usar estas extensiones, generalmente necesitamos herramientas que las soporten explícitamente (como `egrep`, `grep -E` o `sed -r`).

La regla de oro es: **La mayoría de los caracteres son literales**, es decir, concuerdan consigo mismos.

- **Ejemplo:** La regex `casa` busca exactamente la secuencia "c", "a", "s", "a".

La excepción son los **metacaracteres**, símbolos con significado especial que definen el patrón.

Símbolo	Función	Significado	Ejemplo	Coincide con...
<code>.</code>	Comodín	Cualquier carácter (excepto salto de línea).	<code>a.c</code>	"abc", "axc", "a@c"
<code>[ ]</code>	Clase	Cualquiera de los caracteres dentro.	<code>[abc]</code>	"a", "b", "c"
<code>[^ ]</code>	Negación	Cualquier carácter <b>que no esté</b> dentro.	<code>[^abc]</code>	"d", "z", "1" (pero no a, b o c)
<code>^</code>	Ancla	Inicio de línea.	<code>^abc</code>	Líneas que empiezan por "abc"

Símbolo	Función	Significado	Ejemplo	Coincide con...
\$	Ancla	Final de línea.	abc\$	Líneas que terminan en "abc"
\	Escape	Quita el significado especial al siguiente carácter.	\.	Un punto literal "."
	Lógica	Operador OR (alternancia).	gato perro	"gato", "perro"
( )	Grupo	Agrupar patrones y permite capturarlos.	(ab)+	"ab", "abab", "ababab"

**Cuantificadores de repetición:** Estos afectan al carácter o grupo que tienen **inmediatamente a su izquierda**

Símbolo	Significado	Mnemotécnica	Ejemplo a...
*	0 o más veces.	"El opcional múltiple"	a* (vacío, a, aa, aaa...)
+	1 o más veces.	"El obligatorio"	a+ (a, aa, aaa...)
?	0 o 1 vez.	"El opcional único"	a? (vacío o a)
{n}	Exactamente <i>n</i> veces.	Exacto	a{3} (aaa)
{n,}	Al menos <i>n</i> veces.	Mínimo	a{2,} (aa, aaa...)
{n,m}	Entre <i>n</i> y <i>m</i> veces.	Rango	a{2,4} (aa, aaa, aaaa)

Dentro de una clase de caracteres (corchetes), la mayoría de metacaracteres **pierden su significado especial** y se vuelven literales.

- `[a.]c` → Busca una 'a' o un punto literal '.', seguido de 'c'. (Aquí el punto no es "cualquier carácter").

Si quieres buscar literalmente un corchete, un guion o un circunflejo dentro de una clase, su posición importa:

1. **El guion** `-`: Ponlo al final de la lista. `[a-z-]` (letras o un guion).
2. **El cierre** `]`: Ponlo al principio de la lista. `[ ]abc` (un corchete cerrado, a, b, o c).
3. **El circunflejo** `^`: Ponlo en cualquier sitio **menos al principio** (si va al principio niega). `[a^b]` (a, circunflejo, o b).

### Ejemplos:

- `a..c`: Una 'a', dos caracteres cualquiera, una 'c'. (Ej: `azxc`, `a12c`).
- `[a-z]`: Cualquier letra minúscula.
- `ab*c`: Una 'a', cero o muchas 'b', una 'c'. (Ej: `ac`, `abc`, `abbbc`).
- `ab+c`: Una 'a', una o muchas 'b', una 'c'. (Ej: `abc`, `abbc`). **No coincide con `ac`**.
- `.*`: Cualquier cadena de texto (muy común).
- `^#.*\.$`: Línea que es un comentario (empieza por `#`) y termina en un punto literal.
- `(a|b)c`: Puede ser `ac` o `bc`.

- `x(abc)*x`: Una 'x', seguida de la secuencia "abc" repetida 0 o más veces, y otra 'x'. (Ej: xx, xabcx, xabcbcx).

## Conceptos avanzados:

**Etiquetado y Retroreferencias (Backreferences):** Lo que pongas entre paréntesis `( )` se guarda en memoria temporalmente. Puedes "pegar" lo que se encontró usando `\1` (para el primer paréntesis), `\2` (para el segundo), etc.

### Ejemplo de detección:

- Patrón: `(.)oo\1`
- Explicación: Busca cualquier carácter (grupo 1), seguido de "oo", seguido de **lo mismo que se encontró en el grupo 1**.
- Coincide con: `moom`, `noon`.
- No coincide con: `moon` (porque la 'n' final no es igual a la 'm' inicial).

**Ejemplo de sustitución con `sed`:** Podemos reordenar el texto usando las referencias.

```
Intercambiar el orden de dos caracteres
sed -r "s/(.)(.)/\2\1/g"
Si la entrada es "ab", el grupo 1 captura 'a', el grupo 2 captura 'b'.
La salida \2\1 imprime primero 'b' y luego 'a' -> "ba".
```

**El carácter `&` en `sed`:** En la parte de sustitución, el símbolo `&` representa **toda la cadena que coincidió con el patrón**.

```
sed -r "s/hola/[\&]/g"
Si encuentra "hola", lo sustituye por "[hola]".
```

**Concordancia Codiciosa (Greedy Matching):** Las expresiones regulares son "codiciosas" por naturaleza: intentan abarcar la **mayor cantidad de texto posible** que cumpla el patrón.

- Texto: `yyyaxaxayyy`
- Patrón: `a(.*)a` (una 'a', lo que sea, y otra 'a')
- Resultado: Detectará `axaxa` entero.
- *Nota:* No se detendrá en la segunda 'a' (axa), sino que irá hasta la última posible.

## Porrazos

### Ejemplo 1:

```
#concuenda con direcciones de e-mail
\w+@\w+\.\w+((\.\w+)*)?
```

`\w` (word character) es un atajo para no tener que escribir `[a-zA-Z0-9_]`.

- `\w` = Cualquier letra (a-z), número (0-9) o guion bajo (\_).
- `\w+` = Una o más letras/números seguidos.

El concepto avanzado aquí es la **Agrupación Anidada** para estructuras repetitivas.

- **El problema:** Un dominio puede tener una extensión (`.com`), dos (`.co.uk`), tres (`.ptic.mec.es`), etc. No sabemos cuántas.
- **La solución avanzada:** `((\.\w+)*)?`
  1. **Grupo interno** `(\.\w+)`: Define la unidad mínima que se repite (un punto seguido de texto).
  2. **Cuantificador interno** `*`: Dice "esta unidad puede aparecer 0, 1 o mil veces".
  3. **Grupo externo con** `?`: Dice "y todo este lío de repeticiones es, en sí mismo, opcional".

## Ejemplo 2:

```
concuerda con fechas en el formato dd-mm-yyyy (años entre el 1900 y 2099)
(0[1-9] | [12] [0-9] | 3[01])-(0[1-9] | 1[012])-(19|20)[0-9]{2}
```

Este es el ejemplo más importante para un administrador de sistemas. **Las expresiones regulares NO saben matemáticas**. No saben que 32 es mayor que 31. Solo ven caracteres.

Para validar un rango numérico (como los días del 1 al 31), tenemos que **descomponer el número en patrones de texto** usando el operador **OR** (`|`).

Caso	Regex	Explicación	Números que cubre
A	<code>0[1-9]</code>	Empieza por 0, sigue un dígito del 1 al 9.	01, 02... 09
B	<code>[12][0-9]</code>	Empieza por 1 o 2, sigue cualquier dígito.	10, 11... 29
C	<code>3[01]</code>	Empieza por 3, sigue un 0 o un 1.	30, 31
Desglose del AÑO `(19 20)[0-9]{2}`:			

- `(19|20)`: Restringimos el siglo. Solo aceptamos 19 o 20.
- `[0-9]{2}`: Aceptamos cualquier combinación para los dos últimos dígitos.
- **Resultado:** Valida desde 1900 hasta 2099.

## Ejemplo 3:

```
concuerda con números en punto flotante (con o sin exponente)
[+-]?([0-9]*\.)?[0-9]+([eE][+-]?[0-9]+)?
```

- **El Signo Opcional:** `[+-]?`
- **La parte decimal (El truco del asterisco):** `([0-9]*\.)?`



- Aquí hay algo curioso: `[0-9]*` (asterisco, no más). Esto significa "cero o más dígitos".
- ¿**Por qué?** Para permitir escribir `.5` (sin el cero delante). Si pusiéramos `+`, obligaríamos a escribir `0.5`.
- Todo el grupo es opcional `?` porque un entero (50) no lleva punto.

- **La parte obligatoria:** `[0-9]+`

- Al menos debe haber un dígito en algún lado.

- **Notación Científica (Exponente):** `([eE][+-]?[0-9]+)?`

- Acepta `e` o `E`.
- Acepta un signo opcional para el exponente (`e-5`).
- Obliga a que haya dígitos detrás de la 'e' (no vale `10e`).

## 5.2 Procesamiento de textos

En UNIX/Linux, estos comandos se conocen como **filtros**. Su filosofía es recibir datos por la **entrada estándar** (stdin) o de un fichero, procesarlos, y enviar el resultado a la **salida estándar** (stdout).

**Flujo típico:** `entrada` → `filtro` → `salida`

### LA POTENCIA DE LAS TUBERÍAS

Estos comandos brillan cuando se combinan.

Ejemplo: `sort < fichero.txt | head -3 > top3.txt` (Ordena el fichero y guarda las 3 primeras líneas en uno nuevo).

Comando	Función Principal
<b>head / tail</b>	Muestra el principio o final de un fichero.
<b>tac / rev</b>	Invierte el fichero (por líneas o por caracteres).
<b>wc</b>	Cuenta líneas, palabras y bytes ( <i>Word Count</i> ).
<b>sort</b>	Ordena las líneas (alfabética o numéricamente).
<b>uniq</b>	Elimina líneas repetidas <b>consecutivas</b> .
<b>tr</b>	Traduce o elimina caracteres (solo desde stdin).
<b>cut</b>	Corta y extrae columnas (campos) de texto.
<b>paste</b>	Pega ficheros lado a lado (concatena líneas).
<b>join</b>	Cruce de bases de datos (SQL JOIN) en ficheros de texto.
<b>split</b>	Divide un fichero grande en trozos pequeños.

## NOTA

Para el final no hay que saberse ninguna opción

### 5.2.1 Visualización Parcial (`head`, `tail`, `tac`, `rev`)

#### `head` y `tail`

Muestran las primeras o últimas líneas. Son fundamentales para ver logs o cabeceras de archivos.

```
head [opciones] fichero
```

Opción	Descripción
<code>-n X</code>	Muestra X líneas (por defecto son 10).
<code>-f</code>	(Solo <i>tail</i> ) <b>Follow</b> . Se queda esperando nuevos datos (ideal para logs en tiempo real).

#### Ejemplos:

```
Muestra las 2 primeras líneas con cabecera (-v)
$ head -n 2 -v quijote.txt
==> quijote.txt <==
En un lugar de la Mancha, de cuyo nombre
no quiero acordarme, no ha mucho tiempo

Muestra las 2 últimas líneas
$ tail -n 2 quijote.txt
astillero, adarga antigua, rocín flaco y
galgo corredor.
```

#### `tac` y `rev`

- `tac`: Es `cat` al revés. Imprime la última línea primero, luego la penúltima...
- `rev`: Invierte el orden de los caracteres de cada línea (efecto espejo).

```
$ cat archivo.txt
Hola
Mundo

$ tac archivo.txt | $ rev archivo.txt
```

Mundo		aloH
Hola		odnuM

## 5.2.2 Estadísticas y Ordenación (wc, sort, uniq)

### wc (Word Count)

Cuenta elementos en el fichero. Si no se ponen opciones, muestra: líneas, palabras y bytes.

- **Opciones:** `-l` (líneas), `-w` (palabras), `-c` (bytes/caracteres).

```
$ wc -l quijote.txt
5 quijote.txt
```

### sort

Ordena las líneas. **Importante:** Por defecto ordena alfabéticamente (ASCII), donde 10 va antes que 2.

```
sort [opciones] fichero`
```

Opción	Descripción
<code>-n</code>	Orden numérico (para que 2 vaya antes que 10).
<code>-r</code>	Orden inverso (Reverse).
<code>-f</code>	Ignorar mayúsculas/minúsculas (Case insensitive).
<code>-k N</code>	Ordenar por la columna número N.

```
Ordenar ignorando mayúsculas (-f) por la segunda columna (-k 2)
$ sort -f -k 2 nombres.txt
luis Andión
Adriana Gómez
```

### uniq

Filtra líneas repetidas.

### ¡CUIDADO!

uniq solo detecta duplicados si son adyacentes (están juntos). Por eso SIEMPRE se suele usar sort antes que uniq.

Uso típico: sort fichero | uniq

- **Opciones:** `-c` (cuenta las repeticiones), `-u` (muestra solo las únicas), `-d` (muestra solo las repetidas).

```
$ sort nombres.txt | uniq -c
 1 Enrique Pena
 2 Celia Fernández <-- Aparecía 2 veces
```

## 5.2.3 Manipulación de Contenido (`tr`, `cut`)

### `tr` (Translate)

Sustituye o elimina caracteres.

- **Nota:** `tr` **NO** acepta nombres de archivo como argumento. Funciona solo con la entrada estándar (`<` o `|`).

```
Cambiar minúsculas a mayúsculas
$ tr 'a-z' 'A-Z' < quijote.txt

Eliminar (-d) espacios en blanco
$ tr -d ' ' < quijote.txt

Sustituir múltiples caracteres (a->p, u->k)
$ tr 'au' 'pk' < quijote.txt
"En un lugar" se convierte en "En kn lkgpr"
```

### `cut`

Corta columnas verticales de un fichero.

Opción	Descripción
<code>-c 1-5</code>	Corta por posición de carácter (del 1 al 5).
<code>-d 'x'</code>	Define el delimitador (separador) como 'x' (por defecto es tabulador).
<code>-f N</code>	Elige el campo (field) número N.

```
Fichero: Luis Andi3n (separado por espacio)
Queremos solo el nombre (campo 1) usando el espacio como separador
$ cut -d ' ' -f 1 nombres-ord.txt
Luis
Adriana
```

## 5.2.4 Gesti3n de m3ltiples ficheros (`paste`, `join`, `split`)

### `paste` (Concatenaci3n horizontal)

Pega l3neas de dos ficheros una al lado de la otra.

```
$ paste nombres.txt apellidos.txt
Luis Andi3n
Adriana G3mez
```

### `join` (Cruce relacional)

Combina dos ficheros bas3ndose en un **campo com3n** (como una ID o clave).

#### REQUISITO OBLIGATORIO

Para que join funcione, ambos ficheros deben estar ordenados previamente por el campo que se va a usar para unir.

```
Unir fichero1 y fichero2 usando el campo 2 (-j 2) como clave
$ join -j 2 nombres1.txt nombres2.txt
Andi3n Luis Pedro <-- (Clave com3n) (Resto f1) (Resto f2)
```

### `split`

Divide un archivo grande en trozos peque1os.

```
split [opciones] fichero [prefijo_salida]
```

- **Opciones:** `-l N` (cada N l3neas), `-b N` (cada N bytes), `-d` (usar sufijos num3ricos 00, 01... en vez de aa, ab...).

```
Dividir quijote.txt en archivos de 2 l3neas cada uno
$ split -l 2 -d quijote.txt quij_
```

```
Genera: quij_00, quij_01, quij_02...
```

## 5.3 `awk`

`awk` es, posiblemente, la herramienta más potente que existe. A diferencia de `cut` o `grep`, **`awk` es un lenguaje de programación completo** diseñado para procesar datos estructurados (columnas).

**AWK** (Aho, Weinberger y Kernighan) está diseñado para procesar texto organizado en tablas (filas y columnas), como los logs del sistema, `/etc/passwd` o la salida de comandos como `ls -l`.

- **Filosofía:** Lee la entrada **línea a línea**.
- **Automático:** Divide cada línea en **campos** (columnas) automáticamente sin que tengas que decírselo.
- **Sintaxis:** Muy similar al lenguaje C.

### 5.3.1 Estructura de un programa `awk`

Un script de `awk` tiene forma de "sándwich". Tiene tres partes, aunque no es obligatorio usarlas todas:

Sección	Sintaxis	¿Cuándo se ejecuta?	Uso típico
1. Cabecera	<code>BEGIN { ... }</code>	<b>Una sola vez</b> , ANTES de leer la primera línea.	Inicializar variables, definir separadores ( <code>FS</code> ), imprimir títulos.
2. Cuerpo	<code>/patrón/ { ... }</code>	<b>Una vez por cada línea</b> del fichero.	Filtrar datos, cálculos, imprimir columnas.
3. Cierre	<code>END { ... }</code>	<b>Una sola vez</b> , DESPUÉS de leer la última línea.	Imprimir totales, promedios o mensajes de despedida.

Lo más habitual es usarlo directamente en la terminal con comillas simples `'...'` para proteger el código del shell.

```
awk 'BEGIN { setup } /filtro/ { accion } END { conclusion }' fichero
```

### 5.3.2 Manejo de Campos

Cuando `awk` lee una línea, la trocea automáticamente basándose en los espacios o tabuladores (por defecto).

- **`$0`**: La línea completa (sin trocear).
- **`$1`**: El primer campo (columna 1).
- **`$2`**: El segundo campo.
- **`$N`**: El enésimo campo.

**\*\*VISUALIZACIÓN\*\*** IMAGINA LA FRASE: `HOLA MUNDO LINUX`

- `$0` = "Hola mundo linux"
- `$1` = "Hola"
- `$2` = "mundo"

### Ejemplo básico:

```
Imprime la columna 1 y la 3
echo "uno dos tres cuatro" | awk '{ print $1, $3 }'
Salida: uno tres
```

## 5.3.3 Variables de Control

`awk` tiene variables internas que controlan cómo lee y escribe los datos. Las más importantes para un administrador son:

Variable	Significado	Descripción
<code>FS</code>	<i>Field Separator</i>	Qué carácter separa las columnas en la <b>entrada</b> . (Defecto: espacio).
<code>OFS</code>	<i>Output Field Separator</i>	Qué carácter separará las columnas en la <b>salida</b> (al hacer <code>print</code> ).
<code>NR</code>	<i>Number of Records</i>	Número de línea actual (contador de líneas procesadas).
<code>NF</code>	<i>Number of Fields</i>	Número total de columnas que tiene la línea actual.

**Ejemplo vital: Leer `/etc/passwd`** Como este archivo usa dos puntos ( `:` ) y no espacios, debemos cambiar el `FS`.

```
Cambiamos el separador de entrada (FS) a ":"
Cambiamos el separador de salida (OFS) a una flecha "->"
awk 'BEGIN { FS=":"; OFS="->" } { print $1, $6 }' /etc/passwd

Salida:
root->/root
daemon->/usr/sbin
```

## 5.3.4 Filtrado (El "cerebro" del cuerpo)

En la parte central del script, puedes decidir a qué líneas aplicas las acciones.

1. **Sin filtro:** `{ print $1 }` → Afecta a TODAS las líneas.

2. **Con Regex:** `/Error/ { print $0 }` → Solo líneas que contengan "Error".

3. **Con Lógica:** `$3 > 500 { print $1 }` → Solo si el valor de la columna 3 es mayor de 500.

```
df -h | awk '
Si la línea empieza por "/dev/sd" (es un disco físico)...
/^\/dev\/sd/ {
 # Imprime nombre de partición ($1) y porcentaje ($5)
 print "Partición:", $1, "Ocupación:", $5
}'
```

### 5.3.5 Programación Avanzada (Variables y Operaciones)

Al ser un lenguaje similar a C, puedes usar variables, sumas, condicionales (`if`) y formato avanzado (`printf`).

**Ejemplo Completo: Sumar el tamaño de archivos** Queremos sumar el tamaño (columna 5 de `ls -l`) de todos los archivos y mostrar el total al final.

```
ls -l | awk '
BEGIN {
 suma = 0; # Inicializamos variable (opcional, en awk empiezan en 0)
 print "---- Iniciando Suma ----"
}

Cuerpo: Se ejecuta para CADA archivo
{
 suma = suma + $5; # Acumulamos el tamaño
 printf("Archivo: %s - %d bytes\n", $9, $5); # printf tipo C
}

END {
 print "-----";
 print "Total bytes:", suma;
}
'
```

### 5.3.6 Formas de Ejecución

Tienes 3 formas de correr `awk`, igual que `bash` o `python`:

1. **Línea de comandos (One-liner):** Para cosas rápidas.



```
awk '{print $1}' fichero.txt
```

2. **Fichero de script (-f)**: Cuando el código es largo.

```
awk -f mi_programa.awk datos.txt
```

3. **Ejecutable (Shebang)**:

- Creas un fichero con la primera línea: `#!/usr/bin/awk -f`
- Le das permisos: `chmod +x script.awk`
- Ejecutas: `./script.awk datos.txt`

## 5.4 Python

Python es un lenguaje de **tipado dinámico**: no declaras el tipo de variable (como `int a;` en C), sino que el valor asignado define el tipo.

### 5.4.1 Tipos de Datos y Operadores

#### Tipos Básicos

- **Enteros (int)**: Precisión ilimitada (puedes guardar números gigantes).
- **Flotantes (float)**: Equivalente al `double` de C.
- **Cadenas (str)**: Se delimitan por comillas simples `' '` o dobles `" "`.
- **Booleanos (bool)**: `True` o `False` (Ojo, la primera letra va en mayúscula).

#### Entrada y Salida:

- `input()`: Siempre devuelve un **string**. Si quieres números, debes convertirlo (cast).
- `print()`: Imprime cualquier cosa.

```
Conversión de tipos (Casting)
a = int(input("Dame un número: ")) # De str a int
b = str(10.5) # De float a str
```

#### Operadores

Python tiene una sintaxis muy limpia para operaciones matemáticas y lógicas.

Tipo	Operadores	Notas
Aritméticos	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	La división <code>/</code> siempre devuelve float.
División Entera	<code>//</code>	Devuelve solo la parte entera.

Tipo	Operadores	Notas
Resto (Módulo)	%	
Potencia	**	2 ** 3 es 8.
Bit a Bit	(OR), & (AND), ^ (XOR), ~ (NOT), <<, >>	Operaciones binarias a bajo nivel.
Lógicos	and, or, not	Se usan palabras en inglés, no && o

## 5.4.2 Estructuras de Datos (Colecciones)

Esta es la parte más importante para manejar datos. Se dividen en cuatro grandes grupos:

### Listas (list)

Secuencias ordenadas y **mutables** (se pueden cambiar).

- **Definición:** Corchetes []
- **Slicing (Troceado):** lista[inicio:fin]. (El fin no se incluye).
- **Índices negativos:** -1 es el último elemento.

```
frutas = ['naranjas', 'uvas', 123]
frutas.append('peras') # Añadir al final
frutas.pop() # Sacar el último
frutas[0:2] # Elementos 0 y 1 (el 2 no entra)
```

### Tuplas (tuple)

Secuencias ordenadas e **inmutables** (una vez creadas, no se tocan). Son más rápidas y seguras que las listas.

- **Definición:** Paréntesis () (o a veces sin nada, separadas por comas).

### Conjuntos (set)

Colecciones desordenadas de elementos **únicos** (no admite duplicados). Ideales para operaciones matemáticas de conjuntos.

- **Definición:** set() o llaves {} con valores sueltos.

Operación	Símbolo	Descripción
Unión	a   b	Elementos en A o en B (o en ambos).

Operación	Símbolo	Descripción
Intersección	<code>a &amp; b</code>	Elementos que están en A y también en B.
Diferencia	<code>a - b</code>	Elementos de A quitando los que estén en B.

## Diccionarios (`dict`)

Colecciones de pares **Clave-Valor**. Funcionan como una base de datos rápida o una tabla hash.

- **Definición:** Llaves `{ clave: valor }`.

```
user = {'nombre': 'Ana', 'edad': 19}
print(user['nombre']) # Acceder: Ana
user['edad'] = 20 # Modificar
```

### 5.4.3 El problema de las Referencias (Vital)

En Python, las variables son "etiquetas" que apuntan a objetos en memoria.

- **Tipos Simples (int, str):** Si copias `b = a` y cambias `a`, `b` **NO** cambia. (Se crea un nuevo objeto).
- **Colecciones (listas, dict):** Si copias `b = a`, ambas apuntan a la **misma lista**. Si tocas `a`, **cambias** `b`.

¿Cómo copiar de verdad?

Para tener listas independientes, hay que clonarlas explícitamente:

```
lista_a = [1, 2, 3]
lista_b = lista_a[:] # El slicing [:] crea una copia nueva
```

### 5.4.4 Control de Flujo y Funciones

#### Bucles y Condicionales

- `if / elif / else`: Importante la indentación (tabulaciones).
- `for`: Itera sobre colecciones directamente.
- `range(start, stop, step)`: Genera secuencias numéricas.

#### Funciones (`def`)

Python permite argumentos con valores por defecto y argumentos arbitrarios:

- `*args`: Recibe una tupla de argumentos extra (sin nombre).

- **\*\*kwargs**: Recibe un diccionario de argumentos extra (con nombre clave=valor).

### Estructura del Script (`if __name__ == "__main__":`)

Este bloque es crucial en scripts reutilizables.

- **Sin esto**: Si importas el fichero desde otro script, se ejecutaría todo el código.
- **Con esto**: El código dentro del `if` solo se ejecuta si lanzas el script directamente (`python script.py`), pero no si lo importas (`import script`).

## 5.4.5 Programación Orientada a Objetos (POO)

Python permite definir clases para encapsular datos y funciones.

- **Constructor**: `__init__(self, ...)` se ejecuta al crear el objeto.
- **self**: Referencia al propio objeto (como `this` en Java/C++). Debe ser el primer argumento de cada método.
- **Privacidad**: Si una variable empieza por `__` (doble guion bajo), es privada y no se puede acceder desde fuera fácilmente.
- **Herencia Múltiple**: Una clase puede heredar de varias "madres": `class Hija(Madre1, Madre2)`.

```
class Servidor:
 def __init__(self, ip):
 self.ip = ip # Atributo público
 self.__estado = "OFF" # Atributo privado

 def encender(self):
 self.__estado = "ON"
```

# 6. Actividades Administrativas Básicas

## 6.1 Gestión de procesos

Un **proceso** no es más que una instancia de un programa en ejecución. Imagina que el programa (en el disco duro) es una receta de cocina, y el proceso es "estar cocinando esa receta" en este momento.

### 6.1.1 Conceptos Fundamentales

El **Planificador (Scheduler)** del Kernel es el "jefe de cocina". Decide qué proceso entra a la CPU y cuánto tiempo se queda.

- **Proceso vs. Hilo (Thread):**
  - **Proceso:** Es pesado. Tiene su propio espacio de memoria aislado. Si uno falla, los demás suelen seguir vivos.
  - **Hilo (Subproceso):** Es ligero. Viven *dentro* de un proceso y comparten memoria y recursos. Si un hilo corrompe la memoria, puede tumbar todo el proceso.
- **Afinidad de Núcleo:** Mover un proceso de un núcleo (Core A) a otro (Core B) es costoso ("cache miss"). El sistema intenta evitarlo.

### 6.1.2 Ciclo de Vida y Estados

Los procesos no solo están "ejecutándose" o "parados". Tienen un ciclo de vida complejo.

Código	Estado	Significado para el Admin
R	Running	Está en la CPU o en la cola listo para entrar ya.
S	Sleep (Interruptible)	Durmiendo. Espera algo trivial (que teclees algo, un temporizador).
D	Disk Sleep (Uninterruptible)	<b>Peligroso.</b> Espera al Hardware (Disco/Red). No se puede matar con <code>kill -9</code> hasta que el hardware responda.
T	Stopped	Pausado manualmente (Ctrl+Z o señal SIGSTOP).
Z	Zombie	Muerto viviente. El proceso terminó, pero su proceso "padre" no ha leído su estado de salida. No consumen RAM ni CPU, solo un hueco en la tabla.

## 6.1.3 Monitorización: "Ver qué pasa"

---

### Herramientas Estáticas ( `ps` , `pstree` )

`ps` toma una "foto fija" del momento actual.

Sin opciones, `ps` sólo muestra los procesos lanzados desde el terminal actual y con el mismo `EUID` que el usuario que lo lanzó

#### Sintaxis Clave (Trucos):

- **Estilo UNIX (con guion):** `ps -ef` → Muestra **todo** con detalles (formato estándar).
- **Estilo BSD (sin guion):** `ps aux` → Muestra **todo** incluyendo consumo de CPU/MEM y procesos sin terminal.
- **Personalizado:** `ps -eo pid,user,cmd --sort -%mem` (Muestra PID, usuario y comando ordenado por consumo de RAM).

Algunas opciones:

- `-e` : muestra todos los procesos
- `-u usuario` : muestra los procesos de un usuario
- `-o formato` : permite definir el formato de salida, por ejemplo

`pstree` : Muestra la jerarquía. Fundamental para ver quién es el padre de quién (y entender por qué si matas al padre, mueren los hijos).

### Herramientas Dinámicas ( `top` )

`top` es un monitor en tiempo real. En la **cabecera** podemos ver la hora actual, tiempo que el sistema lleva encendido, el número de usuarios conectados y la carga media del sistema para los últimos 1, 5, y 15 minutos. El número total de tareas y resumen por estado y el estado de ocupación de la CPU y la memoria. En un sistema de  $n$  núcleos el máximo de uso de CPU es  $n \times 100 \%$ .

#### DEBUG AVANZADO: STRACE

Si un proceso falla y no sabes por qué, `strace -p PID` te muestra las "tripas": todas las llamadas que el proceso le hace al Kernel (abrir archivos, leer memoria, etc.).

## 6.1.4 Control y Señales: "Mandar órdenes"

---

No "matamos" procesos, les enviamos **señales**. El proceso recibe la señal y decide qué hacer (salvo con SIGKILL).

`kill -l` lista el conjunto de señales

ID	Nombre	Atajo Teclado	¿Qué hace?	Descripción
15	<b>SIGTERM</b>	<code>kill PID</code>	"Termina, por favor"	<b>Recomendada.</b> Permite al proceso guardar datos y cerrar ficheros antes de salir.
9	<b>SIGKILL</b>	<code>kill -9 PID</code>	"¡MUERE!"	<b>Brutal.</b> El Kernel arranca el proceso de la CPU. Puede corromper datos.
2	<b>SIGINT</b>	<code>Ctrl + C</code>	Interrumpir	Cancela el comando actual en terminal.
1	<b>SIGHUP</b>	-	Recargar	Se usa para reiniciar <i>daemons</i> y que releen su configuración sin detenerse.
20	<b>SIGSTOP</b>	<code>Ctrl + Z</code>	Pausa	Detiene el proceso y lo deja en segundo plano (estado T).

## Comandos de Envío

- `kill [señal] PID`: Envía señal a un ID específico.
- `pkill nombre`: Mata procesos buscando por nombre (ej: `pkill firefox`).
- `killall nombre`: Mata *todos* los procesos con ese nombre exacto.

### INFO

Con `pgrep` buscamos en la lista de procesos para localizar el PID a partir del nombre (similar a `ps | grep`) `pgrep sshd # devuelve el PID del proceso sshd de root`

`exec` ejecuta un comando reemplazado al shell desde el que se lanza. Por ejemplo si te tiras en la terminal un `exec ls` tu terminal va a morir, porque se va a convertir en un `ls`.

- **Situación inicial:** Estás sentado. El **Camarero** (tu Shell, digamos `bash`) está esperando una orden.
- **La Orden:** Tú le dices: `exec ls`.
  - Traducido: "Camarero, quiero que dejes de ser camarero y te transformes en el comando 'Listar Archivos'".
- **La Transformación:** El Camarero **desaparece**. En su lugar, aparece el programa `ls` (que es muy simple y rápido).
- **OJO:** Ya no hay Camarero. Solo está `ls`.

## 1. Ejecución normal (sin `exec`): Tú le pides un café al Camarero.

- El Camarero (Shell) llama a un Ayudante (Proceso hijo).

- El Ayudante va a por el café.
- El Camarero se queda esperando en tu mesa hasta que el Ayudante vuelve.
- **Resultado:** Tienes al Camarero Y al Ayudante ocupados. Cuando el ayudante acaba, el Camarero sigue ahí para pedirle otra cosa.

**2. Ejecución con `exec` (El suicidio del camarero):** Tú usas `exec`. Le dices al Camarero: "Conviértete en una Cafetera".

- El Camarero se quita el uniforme, desaparece y **se transforma** en la Cafetera.
- Ya no hay Camarero. Solo hay Cafetera.
- La Cafetera hace el café.
- Cuando el café está listo y la Cafetera se apaga... **¿quién te atiende?** ¡Nadie! El Camarero desapareció para convertirse en Cafetera.
- Por eso, se cierra la ventana. Se acabó el servicio.

## Segundo Plano (Background)

Ideal para scripts largos o tareas que no quieres esperar.

1. Lanzas con `&`: `backup.sh &`
2. Si ya lanzaste, pausas con `Ctrl+Z` y mandas al fondo con `bg`.
3. Recuperas al frente con `fg`.
4. `nohup`: Vital si vas a cerrar la terminal y no quieres que el proceso muera (inmune a SIGHUP).

El comando `jobs` permite ver la lista de comandos en background lanzados desde el shell, así como su estado (`fg` y `bg` pueden actuar sobre uno de los jobs identificándolo por su número).

### 6.1.5 Prioridades: `nice` y `renice`

Linux es "democrático" pero permite favoritismos.

- **Rango:** De **-20** (Máxima prioridad / "Egoísta") a **+19** (Mínima prioridad / "Amable").
- **Por defecto:** Los procesos nacen con **0**.

Comando	Uso	¿Quién puede usarlo?
<code>nice</code>	Al arrancar: <code>nice -n -5 comando</code>	Solo Root puede poner valores negativos (prioridad alta).
<code>renice</code>	Ya ejecutándose: <code>renice 10 -p PID</code>	Usuarios normales solo pueden <i>bajar</i> prioridad (hacerse más <i>nice</i> ).



## INFO

`ulimit` El comando interno de bash `ulimit` permite controlar los recursos de los que dispone un proceso arrancado por el shell. `ulimit [opciones] [limite]`

### 6.1.6 Recursos y el Sistema `/proc`

#### `/proc` y `/sys` (Sistemas de Archivos Virtuales)

No están en el disco duro, están en la **RAM**. Son la ventana para ver los datos del Kernel en vivo.

- `/proc/cpuinfo`: Qué procesador tienes.
- `/proc/meminfo`: Cuánta RAM hay libre.
- `/proc/1234/`: Directorio con toda la info del proceso con PID 1234 (sus ficheros abiertos `fd`, su memoria `maps`).

#### Análisis de Rendimiento (Cheatsheet)

1. ¿Cuánto lleva encendido? → `uptime` (mira el *load average*).
2. ¿Quién consume RAM/CPU? → `top` (o `htop` si está instalado).
3. ¿Tengo memoria libre? → `free -h` (Mira la columna *available*, no solo *free*).
4. ¿Qué hacen los usuarios? → `w`.

## 6.2 Gestión del sistema de ficheros

En UNIX/Linux, la filosofía base es que **todo objeto es un fichero**. Esto incluye desde documentos de texto hasta dispositivos de hardware (donde leer datos es recibir input y escribir órdenes es enviar output).

Tenemos múltiples comandos para trabajar con ficheros y directorios: `ls`, `rm`, `cp`, `mv`, `mkdir`, `rmdir`, `touch`, etc.

### 6.2.1 Tipos de ficheros y operaciones

El sistema define siete tipos distintos. Se pueden identificar usando el comando `file` o mirando el primer carácter de `ls -l`.

Tipo	Carácter (ls -l)	Descripción	Creación / Borrado
Ficheros Regulares	-	Archivos usuales (texto, binarios, imágenes).	<code>touch</code> , <code>cp</code> , <code>vi</code> / <code>rm</code>
Directorios	d	Contenedores de referencias a otros ficheros.	<code>mkdir</code> / <code>rmdir</code> , <code>rm -r</code>
Enlaces Simbólicos	l	Punteros a otros ficheros (accesos directos).	<code>ln -s</code> / <code>rm</code>
Dispositivos Caracteres	c	Hardware con E/S byte a byte (ej. teclado).	<code>mknod</code> / <code>rm</code>
Dispositivos Bloques	b	Hardware con E/S por bloques (ej. discos).	<code>mknod</code> / <code>rm</code>
Tuberías (Named Pipes)	p	Comunicación entre procesos (FIFO).	<code>mknod</code> / <code>rm</code>
Sockets	s	Comunicación de procesos en red.	<code>socket()</code> / <code>rm</code>

**Nota:** El comando `file [nombre_fichero]` analiza el contenido para determinar qué es (ej. PDF, ASCII, PNG).

## 6.2.2 Gestión de Enlaces

Los enlaces permiten acceder a un mismo contenido con diferentes nombres. Se gestionan con el comando `ln`.

### Enlaces Duros (Hard Links)

- **Concepto:** Es un nombre adicional para el **mismo inodo** (referencia física en disco).
- **Características:**
  - Todos los enlaces duros son el fichero original.
  - El fichero no se borra hasta eliminar **todos** sus enlaces.
  - No pueden cruzar particiones (deben estar en el mismo sistema de ficheros).
- **Comando:** `ln destino nombre_enlace`

### Enlaces Simbólicos (Soft Links)

- **Concepto:** Un fichero pequeño que contiene la **ruta** hacia otro fichero.
- **Características:**
  - Si se borra el original, el enlace queda "roto" (apunta a nada).
  - Pueden apuntar a ficheros en otras particiones.

- **Comando:** `ln -s destino nombre_enlace`

## 6.2.3 Atributos de un Fichero

Toda la metainformación se puede consultar con `ls -l`.

Estructura de `ls -l`

Ejemplo: `-rw-r--r-- 2 luis luis 12 Sep 22 20:19 fichero`

1. **Tipo:** 1er carácter (`-`, `d`, `l`...).
2. **Permisos:** Sigüientes 9 caracteres (ej. `rw-r--r--`).
3. **Enlaces:** Número de enlaces duros (o subdirectorios si es un dir).
4. **Usuario:** Propietario (`u`).
5. **Grupo:** Grupo propietario (`g`).
6. **Tamaño:** En bytes (usar `ls -lh` para verlo en KB/MB).
7. **Fecha:** Última modificación (`mtime`).
8. **Nombre:** Hasta 255 caracteres (evitar espacios y especiales).

## Tipos de Fechas (Timestamps)

Linux guarda tres marcas de tiempo para cada fichero:

- **mtime:** Modificación del contenido (por defecto en `ls -l`).
- **atime:** Último acceso/lectura (`ls -l --time=atime`).
- **ctime:** Cambio de estado o metadatos, como permisos (`ls -l --time=ctime`).

## 6.2.4 Permisos y Seguridad

### Operaciones Básicas (r, w, x)

El efecto de los permisos cambia si se aplica a un fichero o a un directorio:

Permiso	En Fichero	En Directorio
<b>Lectura (r)</b>	Abrir y leer contenido.	Listar contenido ( <code>ls</code> ).
<b>Escritura (w)</b>	Modificar o truncar contenido.	Crear, borrar o renombrar ficheros dentro.
<b>Ejecución (x)</b>	Ejecutar como programa/script.	Entrar en el directorio ( <code>cd</code> ).

### Categorías de Usuarios

- **User (u) - El Dueño:**
  - Es el **propietario** del fichero. Generalmente, es quien lo creó.

- *Analogía*: Es tu diario personal. Tú decides quién lo lee.

- **Group (g) - El Equipo:**

- Es un conjunto de usuarios que comparten permisos. Si perteneces al grupo "Contabilidad", podrás ver los archivos de ese grupo.
- *Analogía*: Una pizarra en la sala de reuniones de tu departamento. Tú y tus compañeros de equipo pueden escribir, pero los de otros departamentos no.

- **Others (o) - El Resto del Mundo:**

- Cualquier usuario que **no** seas tú (el dueño) y que **no** pertenezca al grupo del archivo.
- *Analogía*: Gente que pasa por la calle frente a la oficina. Quizás puedan mirar por la ventana (leer), pero no entrar (ejecutar) ni reordenar los muebles (escribir).

## Modificación de Permisos: `chmod`

Solo el propietario o `root` pueden cambiarlos.

**Modo Simbólico** Formato: `quien operacion permiso`

- *Quien*: `u`, `g`, `o`, `a` (all).
- *Op*: `+` (añadir), `-` (quitar), `=` (fijar).
- *Ejemplo*: `chmod u+x archivo` (añade ejecución al dueño).

**Modo Numérico (Octal)** Se suma el valor de los permisos deseados:

- `r` = 4
- `w` = 2
- `x` = 1
- `h` = 0
- *Ejemplo*: `chmod 750 archivo`
  - Usuario (7): 4+2+1 (rwx)
  - Grupo (5): 4+1 (r-x)
  - Otros (0): ---

## Permisos Especiales

Afectan a la ejecución y seguridad del sistema. Funcionan mediante **UID** (User ID) y **GID** (Group ID), los números internos que Linux usa para identificar usuarios y grupos.

**Regla visual (Mayúsculas vs Minúsculas)**: Estos permisos se "superponen" visualmente sobre el permiso de ejecución (`x`).

- **Minúscula (s, t):** El permiso especial está activo **Y** hay permiso de ejecución (x). (Correcto).
- **Mayúscula (S, T):** El permiso especial está activo **PERO NO** hay ejecución. (Suele ser un error o inútil; piensa en **Stop**).

**Ejemplo:** Cuando haces un `ls -l`, tú ves `luis`, pero el sistema internamente está chequeando el número `1001`.

Permiso	Letra	Valor Octal	Descripción
<b>SetUID</b>	s / S	4000	El proceso se ejecuta con los permisos del <b>propietario</b> (ej. <code>passwd</code> ). Se ve en el bloque de <i>User</i> .
<b>SetGID</b>	s / S	2000	<b>Archivos:</b> Se ejecuta con permisos del grupo.   <b>Directorios:</b> Los archivos nuevos <b>heredan el grupo</b> de la carpeta. Se ve en el bloque de <i>Group</i>.
<b>Sticky Bit</b>	t / T	1000	Usado en directorios (ej. <code>/tmp</code> ). Solo el dueño del archivo puede borrarlo, aunque otros tengan permiso de escritura. Se ve en el bloque <i>Others</i> .

- **Fijar:** `chmod u+s file`, `chmod g+s file`, `chmod +t dir`.

**Identificadores Reales vs. Efectivos (RUID vs. EUID)** Aquí es donde entra la "magia" de los permisos en procesos. Cuando ejecutas un programa, este tiene una "identidad".

- **UID Real (RUID):** Es **quién eres realmente**. Es el ID del usuario que lanzó el proceso (tú logueado en la terminal).
- **UID Efectivo (EUID):** Es **con qué permisos estás actuando** en ese momento preciso. Determina qué puedes hacer.

### La Analogía del Actor

- **Real:** Eres el actor (Juan).
- **Efectivo:** Te pones un uniforme de Policía para una escena. Mientras llevas el uniforme (Efectivo), la gente te trata como policía y tienes "permisos" de policía, aunque en realidad sigues siendo Juan.

**El caso práctico: Cambiar tu contraseña** El fichero donde se guardan las contraseñas (`/etc/shadow`) solo puede ser modificado por **root**. Tú (usuario normal) no tienes permiso. ¿Cómo cambias tu contraseña entonces usando el comando `passwd`?

1. Lanzas el comando `passwd`.
2. **RUID:** Eres tú (usuario normal).
3. **Efectivo (EUID):** El comando tiene un permiso especial (**SetUID**) que hace que, momentáneamente, tu "uniforme" sea el de **root**.
4. Como tu ID *efectivo* es root, el sistema te deja escribir en el fichero protegido.
5. Al terminar, el proceso muere y tú sigues siendo un usuario normal.

## Cambio de Propiedad

- `chown usuario fichero`: Cambia el propietario.
- `chgrp grupo fichero`: Cambia el grupo.
- `chown usuario:grupo fichero`: Cambia ambos a la vez.

## 6.2.5 Localización de ficheros

### El comando `find`

Buscan en tiempo real recorriendo el árbol de directorios. **Sintaxis:** `find [ruta] [expresión]`

#### Criterios de búsqueda comunes:

- `-name "patron"`: Por nombre (usar comillas).
- `-type [f/d/l...]`: Por tipo de fichero.
- `-user / -group`: Por propietario.

- `-size [+n/-n]`: Por tamaño (k, M, G).
- `-mtime [+n/-n]`: Por días de modificación.
- `-perm`: Por permisos.

### Acciones y Operadores:

- `-exec comando {} \;`: Ejecuta un comando sobre cada resultado ({} es el fichero).
- Operadores lógicos: `-o` (OR), `!` (NOT), `-a` (AND implícito).

**Ejemplos: A. Buscar por nombre (ignorando mayúsculas/minúsculas):** Imagina que perdiste un informe, pero no sabes si lo guardaste como `Informe.txt`, `informe.pdf`, etc.

```
find /home/luis -iname "informe*"
```

**Explicación:** Busca en la carpeta de luis cualquier fichero que empiece por "informe" (la `i` de `iname` hace que le de igual mayúsculas o minúsculas).

**B. Buscar "ficheros fantasma" (muy pesados):** Quieres limpiar el disco y buscas archivos de más de 100 Megas.

```
find / -size +100M
```

**Explicación:** Busca en todo el sistema (/) ficheros con tamaño mayor (+) a 100 MB.

**C. Buscar por usuario y borrarlos (Cuidado con este):** Un empleado llamado "pepe" se fue y quieres buscar todos sus archivos temporales en `/tmp` y borrarlos automáticamente.

```
find /tmp -user pepe -exec rm {} \;
```

### Explicación:

- Busca en `/tmp`.
- Criterio: pertenecen al usuario `pepe`.
- Acción: `-exec` ejecuta el comando `rm` (borrar).
- `{}`: Linux sustituye estos corchetes por el nombre de cada archivo que encuentra.

**D. Buscar ficheros modificados recientemente:** ¿Qué ficheros se modificaron en los últimos 2 días en la carpeta `/etc`? (Útil para saber si un virus o una actualización tocó algo).

```
find /etc -mtime -2
```

**Explicación:** Busca en configuración (`/etc`) por tiempo de modificación (`mtime`) de menos de 2 días (`-2`).

## Otros comandos de búsqueda

- **which**: Busca ejecutables en las rutas del `$PATH`.
- **whereis**: Busca binarios, fuentes y páginas de manual.
- **locate**: Búsqueda instantánea usando una base de datos indexada (requiere actualización previa).

# 6.3 Gestión de Discos y Particiones

## 6.1.1 Particiones y Sistemas de Ficheros

Para añadir un nuevo disco a un sistema Linux ya instalado, el proceso obligatorio sigue siempre estos tres pasos secuenciales:

1. **Particionamiento:** Dividir el disco físico en secciones lógicas (`fdisk`)
2. **Creación del Sistema de Ficheros (Formateo):** Organizar los datos dentro de la partición (`mkfs`).
3. **Montado:** Conectar esa partición al árbol de directorios para poder usarla (`mount`)

### Paso 1: Creación de Particiones

Antes de guardar datos, debemos definir las fronteras del disco.

**Identificación de Dispositivos.** Los discos se encuentran en el directorio `/dev/`.

- **SATA/SCSI/USB:** `/dev/sdX` (ej. `/dev/sda` es el disco 1, `/dev/sdb` el disco 2).
- **NVMe:** `/dev/nvmeX` (discos SSD modernos).

**Herramientas de Particionamiento** El comando `fdisk` es la herramienta clásica. Requiere permisos de administrador (`root`).

- **Sintaxis:** `fdisk [opciones] dispositivo`
- **Opción clave:** `-l` (lista la tabla de particiones actual)
  - *Ejemplo:* `fdisk -l /dev/sdb` (muestra particiones `sdb1`, `sdb2...`).

Si no se le pone opciones abre un menú interactivo para poder borrar, crear, listar, etc.

El comando `parted`, que es una herramienta de GNU más avanzada. Permite:

- Crear y borrar particiones



- **Redimensionar** (cambiar tamaño)
- Copiar y comprobar particiones

[!Lección de Historia for fri] Imagina que **Unix** es el **latín** de los sistemas operativos: es la "lengua madre" de la que derivan casi todos los modernos (Linux, macOS, Android, iOS...), excepto Windows.

Unix nació a finales de los 60 en los **Bell Labs** (propiedad de la compañía telefónica AT&T). Sus creadores principales fueron **Ken Thompson** y **Dennis Ritchie** (quien también creó el lenguaje de programación **C**).

Hasta ese momento, cada ordenador tenía su propio sistema operativo incompatible. Unix trajo tres ideas:

1. **Multiusuario y Multitarea:** Varias personas podían usar la máquina a la vez.
2. **Portabilidad:** Estaba escrito en lenguaje C, así que se podía instalar en diferentes máquinas (no solo en una marca).
3. **La Filosofía Unix:** "Haz una cosa y hazla bien".
  - En lugar de programas gigantes, usa programas pequeños (`ls`, `cp`, `rm`) que se unen con tuberías (`|`).

Al principio, Unix se compartía libremente entre universidades. Pero cuando AT&T se dio cuenta de que tenía una mina de oro, **cerró el código**.

- **Unix se volvió "Software Propietario":**
- Costaba miles de dólares la licencia.
- No podías ver cómo estaba hecho (código cerrado).
- No podías modificarlo ni compartirlo con tu vecino.
- Empresas como HP, IBM y Sun Microsystems crearon sus propias versiones de Unix (HP-UX, AIX, Solaris), incompatibles entre sí. Aquí entra **Richard Stallman**. Él trabajaba en el MIT y estaba acostumbrado a la cultura de compartir código. Cuando vio que Unix y las empresas cerraban el software, se enfadó.

Decidió crear un sistema operativo que fuera:

1. **Técnicamente igual a Unix:** Porque Unix era bueno, estable y todos sabían usarlo (los mismos comandos `ls`, `cd`, `grep`...).
2. **Filosóficamente opuesto:** Totalmente libre y gratuito.

**El nombre lo dice todo:** Llamó a su proyecto **GNU** = GNU's Not Unix.

- *Traducción:* "Voy a hacer un sistema que funciona y se siente exactamente igual que Unix, pero **NO** tiene ni una sola línea de código de AT&T original, por lo que es legalmente libre".

Para visualizarlo mejor, imagina una receta de cocina secreta y famosa:

1. **Unix (La receta original):** Es una hamburguesa deliciosa, pero la receta es secreta, carísima y solo la venden en restaurantes de lujo (Servidores empresariales, Mainframes).
2. **GNU (El libro de cocina libre):** Stallman dijo: "Voy a escribir una receta que sepa igual que la hamburguesa Unix, pero la escribiré desde cero y regalaré el libro". GNU creó el pan, la lechuga, el tomate (las herramientas: compiladores, editores, bash).
3. **Linux (La carne):** A GNU le faltaba la pieza central (el Kernel). Linus Torvalds creó esa pieza en 1991. Aunque el Unix original "puro" casi ha desaparecido, su legado está dividido en dos familias:

|Familia|Estado|Ejemplos| |---|---|---| |**Unix Certificado (Descendientes directos)**|Código cerrado, comercial, caro.|**macOS** (sí, el de Apple es un Unix certificado), **Solaris**, **AIX** (IBM), **HP-UX**.| |**Tipo Unix (Clones Libres)**|Código abierto, imitan a Unix.|**GNU/Linux** (Ubuntu, Debian, RedHat, Android), **FreeBSD**.|

## Paso 2: Creación del sistema de Ficheros

Una vez creada la partición (ej. `/dev/sdb1`), está "cruda". Hay que darle una estructura (NTFS, EXT4, etc.) para guardar archivos.

El comando `mkfs` (Make FileSystem) es un "front-end" (interfaz) que llama a comandos específicos según el tipo deseado.

- **Sintaxis:** `mkfs.tipo [opciones] dispositivo`

Comando Específico	Sistema de Ficheros	Uso Habitual
<code>mkfs.ext4</code> (o <code>ext2/3</code> )	<b>Linux</b>	El estándar actual en Linux.
<code>mkfs.vfat</code> / <code>mkfs.exfat</code>	<b>Windows</b>	Compatibilidad con USBs/Windows.
<code>mkfs.ntfs</code>	<b>Windows</b>	Discos duros de Windows.

**Caso Especial: La Partición Swap (Intercambio).** La memoria Swap actúa como RAM virtual. No usa `mkfs` normal, tiene sus propios comandos.

- **Crear (Formatear):** `mkswap /dev/sdb2`, prepara la partición y asigna un UUID.
- **Activar:** `swapon /dev/sdb2`, hace que el sistema la empiece a usar
- **Verificar:** `swapon -s`, muestra el tamaño y prioridad de las swap activas

## Paso Intermedio: Identificación Única (UUID)

En linux, los nombres `/dev/sda` pueden cambiar si cambias los cables de sitio. Para evitar errores usamos el **UUID** (Identificador Único Universal), que es como la "matrícula" fija de la partición.

- **Comando:** `blkid` (Block ID)
- **Uso:** muestra el UUID y el tipo de sistema de archivos
  - *Ejemplo:* `dev/sda1: UUID="b0f7f038..." TYPE="ext4"`

### Paso 3: Montado (Mounting)

Para acceder a los datos, la partición debe "colgarse" (montarse) de un directorio existente.

#### Montaje Manual (`mount` / `unmount`):

- **Montar:** `mount [opciones] dispositivo directorio_destino`
  - *Ejemplo:* `mount /dev/sdb1 /home2`
- **Desmontar:** `unmount directorio`
  - *Ejemplo:* `unmount /home2`
- **Ver montados:** El fichero `/etc/mtab` contiene la lista en tiempo real de lo que está montado actualmente

#### NOTA

Si montas un dispositivo sobre un directorio que ya existía, si el directorio tenía archivos, no los vas a ver porque este nuevo dispositivo montado los va a "eclipsar", pero no se van a borrar, una vez lo desmontes se volverán a ver.

**Montaje Persistente (`etc/fstab`):** Para que los discos se monten solos al encender el PC, se configuran en este fichero. **Estructura de las columnas en `fstab`:**

1. **File System:** Dispositivo (`/dev/sdb1`) o mejor su **UUID** (`UUID=...`).
2. **Mount Point:** Carpeta donde aparecerá (`/home`, `/`, `none` para swap).
3. **Tipo:** `ext4`, `swap`, `auto`.
4. **Opciones:** Reglas de montaje (separadas por comas).
  - `rw` / `ro`: Lectura-Escritura o Solo-Lectura.
  - `auto` / `noauto`: ¿Se monta al arrancar?
  - `user` / `nouser`: ¿Un usuario normal puede montarlo?
  - `defaults`: Equivale a `rw, suid, dev, exec, auto, nouser, async`.
5. **Dump:** (0 o 1) Para copias de seguridad antiguas (usualmente 0).
6. **Pass:** (0, 1, 2) Orden de chequeo de errores al inicio (0 = no chequear).

Si un directorio aparece listado en el `fstab` puede montarse sin especificar el dispositivo.

### Mantenimiento y Monitorización

Una vez el sistema está funcionando, el administrador debe vigilar el espacio y la salud de los datos.

**Espacio en Disco (`df` vs `du`):** Es vital no confundirlos:

Comando	Significado	Qué mide	Opción Clave
<code>df</code>	Disk Free	Muestra espacio libre/usado de <b>particiones montadas</b> .	<code>-h</code> (human readable: GB, MB).

Comando	Significado	Qué mide	Opción Clave
<code>du</code>	Disk Usage	Muestra espacio ocupado por <b>archivos y carpetas</b> .	<code>-h</code> (human), <code>-s</code> (summary/resumen total).

- *Ejemplo* `df -h`: Veo que mi disco duro está al 90% de capacidad.
- *Ejemplo* `du -hs /home`: Veo cuánto pesa exactamente la carpeta de usuarios.

**Salud del Sistema de Ficheros ( `fsck` )**: Si el sistema se apaga mal, los ficheros pueden corromperse.

- **Comando:** `fsck.tipo [opciones] dispositivo`
  - *Ejemplos:* `fsck.ext4`, `fsck.vfat`.
- **Opción clave:** `-y` (Responde "Yes" a todas las reparaciones automáticamente).
- **Importante:** Nunca ejecutar `fsck` sobre una partición montada (podrías romper datos).

## 6.3.2 Sistemas de ficheros LVM

LVM es una capa de abstracción entre el disco físico y el sistema de ficheros. Rompe la rigidez de las particiones tradicionales.

### Ventajas principales:

- **Flexibilidad:** Permite distribuir el espacio disponible sin depender de la ubicación física en el disco.
- **Redimensionado en caliente:** Puedes aumentar o reducir volúmenes sin apagar la máquina (especialmente útil en servidores).
- **Agregación de espacio:** Si añades un disco nuevo, puedes sumarlo a tu grupo y "estirar" tus particiones actuales para que ocupen ese nuevo espacio.

### Arquitectura y Jerarquía LVM

LVM se organiza en capas, desde lo físico (abajo) hasta lo lógico (arriba).

#### Las Capas:

1. **Discos/Particiones Físicas:** El hardware real (ej. `/dev/sda1`, `/dev/sdb`).
2. **PV (Physical Volume - Volumen Físico):** Es la etiqueta que se le pone a una partición para decirle a Linux "esto será usado por LVM".
3. **VG (Volume Group - Grupo de Volúmenes):** Es una "bolsa de almacenamiento". Agrupa varios PVs en un solo gran almacén de espacio.
4. **LV (Logical Volume - Volumen Lógico):** Son las particiones virtuales que creamos sacando espacio de la "bolsa" (VG). Es el equivalente a `/dev/sda1` en el mundo clásico.
5. **Sistema de Ficheros:** Formateo final (`mkfs`) sobre el LV.

**Unidades de Medida:** LVM no trabaja bit a bit, sino en bloques:

- **PE (Physical Extent):** Unidad básica en la que se divide el Volumen Físico.
- **LE (Logical Extent):** Unidad básica del Volumen Lógico.
- **Relación:** Generalmente 1 LE = 1 PE.
- **Mapeado:** La forma en que los LE se guardan en los PE puede ser:
  - *Lineal:* Uno detrás de otro.
  - *Stripping:* Datos distribuidos (mayor rendimiento).
  - *Mirroring:* Espejo (redundancia).

## Comandos de Gestión LVM

Los comandos siguen una lógica de nombres muy clara según la capa que gestionen: `pv...`, `vg...`, `lv...`.

**Comandos de Información (Montorización):** Para ver qué tenemos configurado.

Capa	Comando Detallado	Comando Resumido
Physical Volume	<code>pvdisk</code>	<code>pvs</code>
Volume Group	<code>vgdisk</code>	<code>vgs</code>
Logical Volume	<code>lvdisk</code>	<code>lvs</code>

**Comandos de Operación (Crear y Modificar):**

- **Gestión de Volúmenes Físicos (PV):** Antes de nada, marcamos la partición para usarla
  - **Crear:** `pvcreate /dev/sdc1`
- **Gestión de Grupos de Volúmenes (VG):** Agrupamos los discos
  - **Crear:** `vgcreate [NombreVG] [Disco1] [Disco2]`
  - **Borrar:** `vgremove [NombreVG]`
  - **Ampliar (Añadir disco):** `vgextend [NombreVG] [NuevoDisco]`
  - **Reducir (Quitar disco):** `vgreduce [NombreVG] [DiscoAQuitar]`
- **Gestión de Volúmenes Lógicos (LV):** Creamos las particiones útiles.
  - **Crear:** `lvcreate -L[Tamaño] -n [NombreLV] [NombreVG]`
    - Ej: `lvcreate -L4.20G -n testlv NuevoGrupo`
  - **Borrar:** `lvremove /dev/GrupoVolmen/testlv` (¡Desmontar antes!)
  - **Ampliar (Extend):**
    - Por tamaño fijo: `lvextend -L12G ...` (Fija el total a 12GB).
    - Sumando espacio: `lvextend -L+1G ...` (Añade 1GB al actual).
    - Por Extents: `lvextend -l+200 ...` (Añade 200 bloques lógicos).
  - **Reducir:** `lvreduce` (Misma sintaxis que extend, pero reduce).

## Uso del Volumen Lógico

Una vez creado el LV, el sistema operativo necesita saber cómo acceder a él

**Nomenclatura de Dispositivos:** Hay dos formas de llamar a un volumen lógico (ambas llevan al mismo sitio):

- **Directa:** `/dev/NombreVG/NombreLV` (ej. `/dev/GrupoVolumen/homelv`).
- **Device Mapper:** `/dev/mapper/NombreVG-NombreLV`.
  - *Nota:* El **Device Mapper** es el componente del kernel que hace la magia de mapear bloques físicos a virtuales. También se usa para cifrado (`dm-crypt`).

**Dar Formato y Montar:** Igual que una partición normal

- **Formatear:** `mkfs.ext4 /dev/GrupoVolumen/homelv`
- **Montar:** `mount /dev/GrupoVolumen/homelv /home`
- **Persistencia:** Añadir al `/etc/fstab`.

## Redimensionado del Sistema de Ficheros

Si agrandas un LV (`lvextend`), el sistema de ficheros que hay dentro (ext4, xfs) no se entera automáticamente. Tienes que "estirarlo" también.

**Comando `fsadm`:** Es una herramienta genérica que chequea y redimensiona.

- **Sintaxis:** `fsadm resize [dispositivo] [nuevo_tamaño]`
- **Ej:** `fsadm resize /dev/mapper/Grupo-vol 2048M` (Si no pones tamaño, ocupa todo el disponible).

**Regla de Oro del Redimensionado:**

1. Para **Agrandar:** Primero el LVM (`lvextend`), luego el sistema de ficheros (`fsadm` o `resize2fs`).
2. Para **Reducir:** Primero el sistema de ficheros (peligroso), luego el LVM (`lvreduce`).

## 6.4 Gestión de Usuarios y Grupos

### 6.4.1 Conceptos Básicos de la Cuenta Unix

En Linux, nadie entra sin una cuenta. Una cuenta no es más que una colección de atributos lógicos que definen **quién eres** y **qué puedes hacer**.

**Componentes de una cuenta**

- **Username (Login):** Nombre único (ej: `pepe`).
- **UID (User ID):** Identificador numérico único. El sistema usa esto, no el nombre.
- **GID (Group ID):** Identificador del grupo principal.

- **Password:** La credencial de acceso.
- **Home Directory:** Tu "casa" en el sistema (ej: `/home/pepe`).
- **Shell:** El intérprete de comandos por defecto (ej: `/bin/bash`).

## Tipos de Usuarios

Tipo	UID Típico	Descripción
<b>Root (Superusuario)</b>	0	Dios del sistema. Acceso total.
<b>Cuentas de Servicio</b>	1 - 999	Usuarios "fantasma" para demonios (ej: <code>apache</code> , <code>lp</code> , <code>mail</code> ). Aumentan la seguridad aislando procesos.
<b>Usuarios Normales</b>	1000 - 65535	Personas reales. Tienen restricciones.

## 6.4.2 Almacenamiento de Información (Los Ficheros)

Toda la gestión de usuarios reside en archivos de texto plano en `/etc/`

### `/etc/passwd` (Información Pública)

Define a los usuarios. Todo el mundo puede leerlo. **Formato:** `usuario:x:UID:GID:GECOS:home:shell`

- `pepe`: Nombre de usuario.
- `x`: Indica que la contraseña está oculta en `/etc/shadow`.
- `1002`: UID.
- `1002`: GID principal.
- `GECOS`: Información extra (Nombre completo, teléfono...).
- `/home/pepe`: Ruta del directorio personal.
- `/bin/bash`: Shell de inicio.

### `/etc/shadow` (Seguridad / Contraseñas)

Solo `root` puede leerlo. Contiene las contraseñas cifradas y datos de expiración. **Formato:**

`usuario:password_cifrado:días_cambio:...`

- Si en el password hay `!` o `*`, la cuenta está bloqueada (no puede hacer login).
- Guarda fechas de caducidad, días de aviso, etc.

### Grupos (`/etc/group` y `/etc/gshadow`)

- `/etc/group`: Define qué usuarios pertenecen a qué grupos.
  - Formato: `nombre_grupo:x:GID:usuario1,usuario2`
- `/etc/gshadow`: Contraseñas de grupo (raramente usadas) y administradores de grupo.

## 6.4.3 Gestión de Cuentas: Creación y Modificación

---

### Método Manual ("The Hard Way")

Útil para entender qué ocurre "bajo el capó". Pasos:

1. Editar `/etc/passwd` (usar `vipw` para bloqueo seguro).
2. Editar `/etc/shadow` (usar `vipw -s`).
3. Editar `/etc/group`.
4. Crear directorio home (`mkdir`).
5. Copiar ficheros base desde `/etc/skel` (plantilla de inicio).
6. Ajustar permisos y dueños (`chown`, `chmod`).
7. Asignar contraseña (`passwd`)

### Comandos de Bajo Nivel (Estándar Linux)

Son universales pero requieren muchas opciones manuales.

- `useradd`: Crea el usuario (a veces inhabilitado por defecto si no se pasan flags).
- `usermod`: Modifica (cambiar shell, grupos, home).
- `userdel`: Borra el usuario.
- `groupadd` / `groupmod` / `groupdel`: Gestión de grupos.

### Comandos de Alto Nivel (Debian/Ubuntu)

Son scripts más amigables que hacen preguntas interactivas.

- `adduser`: Pide contraseña, datos GECOS y crea el home automáticamente.
- `deluser`: Borra usuario y puede preguntar si borrar el home.

### Gestión Masiva

- `newusers`: Crea múltiples usuarios desde un fichero de texto.
- `chpasswd`: Actualiza contraseñas en lote (formato `user:pass`).

## 6.4.4 Gestión de Contraseñas y Seguridad

---

### Comandos Clave

- `passwd [usuario]`: Cambia la contraseña.
  - `-e`: Fuerza al usuario a cambiarla en el próximo inicio de sesión.
- `gpasswd`: Para cambiar la contraseña de un grupo
- `chage`: Gestiona la caducidad (expiration) de la contraseña.



- *Ejemplo:* Obligar a cambiar la clave cada 90 días.
- `mkpasswd`: Genera un hash cifrado de una cadena (útil para scripts).

## Cambiar de Identidad ( `su` vs `sudo` )

- `su [usuario]` (Switch User):
  - Cambia tu sesión a la de otro usuario.
  - Si no pones usuario, asume `root`.
  - Requiere saber la **contraseña del destino** (la de root).
- `sudo [comando]` (SuperUser DO):
  - Ejecuta un comando con privilegios de otro (generalmente root).
  - Requiere la **contraseña del propio usuario** (no la de root).
  - Se configura en `/etc/sudoers` (editar siempre con `visudo` para evitar romper el sistema).

## 6.4.5 Módulos PAM (Pluggable Authentication Modules)

---

Es el "portero" universal de Linux. Es una librería que usan los programas ( `login` , `ssh` , `su` ) para saber si dejarte pasar o no.

### Los 4 tipos de control:

1. **Auth:** ¿Eres quien dices ser? (Pide contraseña, huella, etc.).
2. **Account:** ¿Tienes permiso *ahora*? (Horario permitido, cuenta no caducada).
3. **Password:** Gestión del cambio de clave (complejidad mínima, no repetir la anterior).
4. **Session:** Qué hacer antes/después de entrar (montar directorios, logs).

## 6.4.6 Cuotas de Disco

---

Sistema para evitar que un usuario llene el disco duro.

### Tipos de Límites

- **Límite Suave (Soft):** Puedes pasarte temporalmente.
- **Límite Duro (Hard):** No puedes escribir ni un byte más.
- **Período de Gracia:** Tiempo que tienes para volver por debajo del límite suave antes de que se convierta en duro.

### Implementación

1. Añadir opción `usrquota` o `grpquota` en `/etc/fstab`.
2. Remontar partición.
3. `quotacheck`: Crear los archivos de base de datos de cuotas.

4. `quotaon` : Activar el sistema.
5. `edquota [usuario]` : Editar los límites (abre un editor de texto).
6. `repquota` : Ver informe de uso.

## 6.5 Gestión de redes de área local

### 6.5.1 Fundamentos y Nomenclatura de Interfaces

Linux abstrae el hardware de red en "interfaces". No importa si es cobre, fibra o aire, el sistema lo ve como un dispositivo lógico.

#### INFO

En informática, una **interfaz** es el punto de conexión entre dos cosas distintas. En redes, es el **intermediario** entre tu sistema operativo (Linux) y el mundo exterior (el cable o el Wi-Fi).

**La analogía de la casa:** Imagina que tu ordenador es una **casa**.

- Los datos son **personas** dentro de la casa que quieren salir.
- La red (Internet/LAN) es la **calle**.

Para salir a la calle, necesitas **puertas**. Las interfaces son esas puertas.

- `eth0` (**Ethernet**): Es la puerta principal.
- `wlan0` (**Wi-Fi**): Es la puerta trasera.
- `lo` (**Loopback**): Es como un espejo dentro de la casa. Si le hablas, te respondes a ti mismo (se usa para pruebas internas).

Linux no "habla" directamente con el cable de cobre; le da los datos a la interfaz `eth0`, y ella se encarga de traducirlos a electricidad.

### Nombres de las Interfaces (NICs)

La forma de llamar a las tarjetas de red ha evolucionado:

- **Esquema Clásico (Antiguo):** Nombres simples secuenciales.
  - `eth0`, `eth1` : Ethernet (cable).
  - `wlan0` : Wi-Fi.
  - `ppp0` : Conexiones punto a punto (módem/VPN).
- **Esquema Predecible (Nuevo):** Nombres basados en la ubicación física para evitar que cambien al reiniciar.

- Ejemplo: `enp3s0` (Ethernet, Bus PCI 3, Slot 0).

- **Loopback (`lo`):**

- Es una interfaz virtual de "circuito cerrado".
- IP estándar: `127.0.0.1`.
- Uso: Pruebas internas y comunicación entre procesos de la misma máquina.

## 6.5.2 Archivos de Configuración Clave

En Linux (específicamente basado en Debian/Ubuntu según tu texto), la red se configura mediante ficheros de texto que lee el servicio `networking` al arrancar.

Archivo	Función
<code>/etc/network/interfaces</code>	Configuración principal de las tarjetas (IP, máscara, gateway).
<code>/etc/resolv.conf</code>	Configuración de los servidores DNS (quién resuelve los nombres).
<code>/etc/hosts</code>	"DNS local". Asocia Nombres <-> IPs manualmente. Tiene prioridad sobre el DNS externo.
<code>/etc/hostname</code>	Contiene el nombre de la máquina.

### Configuración Estática vs. Dinámica

Se define en `/etc/network/interfaces`.

**A. Estática (Manual)** Tú defines todos los valores. Ideal para servidores.

```
auto eth0
iface eth0 inet static
 address 193.144.84.77 # Tu IP
 netmask 255.255.255.0 # Máscara de subred
 gateway 193.144.84.1 # Puerta de enlace (salida a internet)
```

**B. Dinámica (DHCP)** Un servidor externo te asigna la configuración.

```
auto eth0
iface eth0 inet dhcp
```

**Nota:** Se puede forzar la petición DHCP manualmente con el comando `dhclient eth0`.

## Configuración de DNS ( `/etc/resolv.conf` )

Define a quién preguntar para traducir `google.com` a una IP.

```
search usc.es # Si buscas "servidor", probará "servidor.usc.es"
nameserver 8.8.8.8 # Servidor DNS 1
nameserver 8.8.4.4 # Servidor DNS 2 (Backup)
```

**Atención:** Si usas DHCP, este archivo suele sobrescribirse automáticamente con lo que diga el servidor DHCP.

## 6.5.3 Comandos de Gestión y Configuración

### `ifconfig` (Interface Configuration)

Muestra el estado o configura la IP temporalmente (se pierde al reiniciar).

- **Lectura ( `ifconfig eth0` ):**
  - **UP / RUNNING:** La tarjeta está encendida y tiene cable conectado.
  - **MTU:** Tamaño máximo del paquete (v.g. 1500 bytes).
  - **RX / TX:** Estadísticas de paquetes recibidos/transmitidos y errores.
- **Escritura:**
  - `ifconfig eth0 192.168.1.5 netmask 255.255.255.0 up`
- **Wireless:** Para Wi-Fi se usa su primo hermano `iwconfig` (ej: `iwconfig eth1 essid "MiWifi"`).

### `route` (Tabla de Enrutamiento)

Decide por dónde enviar los paquetes.

```
$ route -n
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.1.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
0.0.0.0 192.168.1.1 0.0.0.0 UG 0 0 0 eth0
```

**1. Destination (¿A dónde quieres ir?)** Es la dirección final del paquete. Puede ser una dirección concreta o una red entera.

- `192.168.1.0`: Significa "Cualquier ordenador de mi red local (mi casa)".
- `0.0.0.0`: Es un comodín. Significa "Cualquier otro sitio que no esté listado arriba" (es decir, Internet).

**2. Gateway (¿Quién me ayuda?)** Esta es la columna más confusa, pero la más importante.

- Si pone `0.0.0.0` (o `*`): Significa "Nadie". El destino está cerca, conectado directamente a mí por un cable. Puedo gritar y me oyen. No necesito ayuda.
- Si pone una IP (ej. `192.168.1.1`): Significa que el destino está lejos (en Internet). Yo no sé llegar. Tengo que enviarle el paquete a ese "Gateway" (tu Router) para que él se encargue de llevarlo a Google, Facebook, etc.

**3. Iface (¿Por qué puerta salgo?)** Por qué tarjeta física va a salir el dato (`eth0`, `wlan0`...).

**4. Flags:** - `U` (Up): Ruta activa. - `G` (Gateway): Usa un router intermediario. - `H` (Host): Ruta a una sola máquina, no a una red.

La estructura general para modificar rutas es: `route [add|del] [default] [-net|-host] destino [netmask máscara] [gw pasarela] [dev interfaz]`

### `netstat` (Estadísticas de Red)

Muestra qué puertos están abiertos y quién está conectado.

- **Estados:**
  - `ESTABLISHED`: Conexión activa transmitiendo datos.
  - `LISTEN`: Servidor esperando conexiones (ej. un servidor Web).
- **Uso:** `netstat -s` (estadísticas por protocolo: TCP, UDP, ICMP).

## 6.5.4 Diagnóstico y Resolución de Problemas

Comando	Función	Protocolo	Notas
<code>ping [ip]</code>	Comprueba si hay conexión. Mide el tiempo (RTT).	ICMP	Los firewalls pueden bloquearlo.
<code>tracert [ip]</code>	Muestra el camino (saltos) hasta el destino.	UDP + TTL	Si sale <code>*</code> , ese router no responde o tiene firewall.
<code>host</code> / <code>dig</code>	Pregunta al DNS la IP de un nombre.	DNS	Útil para saber si falla la red o falla el nombre.
<code>arp</code>	Muestra la tabla IP <-> MAC.	ARP	Útil para ver conflictos de IP o problemas de hardware local.

#### INFO

Un firewall es un sistema (puede ser un programa o un aparato físico) que **monitoriza y controla** el tráfico de red entrante y saliente. Se coloca entre tu red interna (de confianza) y una red externa (Internet, no confiable).

**La Analogía del Portero de Discoteca:** Imagina que tu ordenador es una discoteca exclusiva.

- **La Interfaz:** Es la puerta de entrada.
- **El Firewall:** Es el portero.
- **Las Reglas:** El portero tiene una lista.
  - "Si viene Luis, déjalo pasar." (Allow)
    - "Si viene alguien con zapatillas, no lo dejes pasar." (Deny/Drop)

## 6.5.5 Conceptos Avanzados

### IP Forwarding (Convertir PC en Router)

Por defecto, Linux descarta paquetes que no son para él.

Si activamos el IP Forwarding (ip\_forward), Linux cogerá paquetes que entran por una tarjeta y los reenviará por otra. Esto convierte al ordenador en un router.

#### **ip** (El comando moderno)

**ip** unifica a **ifconfig**, **route** y **netstat**. Es más potente y complejo, y es el estándar actual aunque en estos apuntes nos hemos centrado en los clásicos.